

Minimizing fStress by Majorizing A Gauss-Newton Approximation

Jan de Leeuw

April 11, 2026

We generalize the usual least squares Multidimensional Scaling loss function stress by adding a monotone weighting function f that transforms both dissimilarities and distances. This defines fStress. The algorithm to minimize fStress first linearizes f at the current distances and then minimizes the loss function with linearized distances with the standard majorization algorithm. Code in both R and C is provided, for both metric and non-metric (ordinal) scaling. Running time comparisons show the C version is 10 to 50 times faster than the R version.

Table of contents

1	Introduction	4
1.1	Metric fStress	5
1.2	Non-metric fStress	6
2	Algorithm	7
2.1	Normalized Cone Regression	7
2.2	Majorization	7
3	Software	11
3.1	Data	11
3.2	Initial Estimate	11
3.3	Functions	12
3.4	Arguments	12
3.5	Value	13

3.6	Plots	13
3.6.1	smacofConfigurationPlot()	14
3.6.2	smacofShepardPlot()	14
3.6.3	smacofDistDhatPlot()	14
3.6.4	smacofResidualPlot()	15
4	Example	16
4.1	Functions	16
4.2	Timings	16
4.3	Plots	17
	References	48

Note: This is a working manuscript which will be expanded/updated frequently. All suggestions for improvement are welcome. All Rmd, tex, html, pdf, R, and C files are in the public domain. Attribution will be appreciated, but is not required. The files can be found at <https://github.com/deleeuw/rStress>

1 Introduction

Multidimensional Scaling (MDS) minimizes the loss function *stress*, originally proposed by Kruskal (1964a), Kruskal (1964b). In *metric* MDS stress is

$$\sigma(x, \delta) := \sum_{k=1}^K w_k (\delta_k - d_k(x))^2, \quad (1)$$

with

- w is a vector of K known positive *weights*;
- δ is a vector of K known non-negative *dissimilarities*;
- $x := \text{vec}(X)$, with X the *configuration*, an $n \times p$ matrix of n *points* in p *dimensions*;
- $d(x)$ are K Euclidean *distances* between some or all pairs of points.

Our notation is slightly non-standard, because we use vectors in our definition and not matrices. For each $1 \leq k \leq K$ there is a pair of indices $(i(k), j(k))$ with $1 \leq i(k) < j(k) \leq n$. The unit vectors $e_{i(k)}$ and $e_{j(k)}$ are used to define the matrix A_k , which is the direct sum of p copies of $(e_{i(k)} - e_{j(k)})(e_{i(k)} - e_{j(k)})'$. Suppose, for example, that $n = 4, p = 2$ and $(i(k), j(k))$ is $(2, 4)$. Then

$$A_k = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & +1 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & +1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & +1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & +1 \end{pmatrix}. \quad (2)$$

The A_k , together with $x = \text{vec}(X)$, allow us to conveniently write $d_k(x) = \sqrt{x' A_k x}$.

In *non-metric* MDS stress is a function of both x and δ , and we minimize over both these arguments. Thus

$$\sigma(x, \delta) = \sum_{k=1}^K w_k (\delta_k - d_k(x))^2$$

We require that $\delta \in \Delta$, where Δ is some subset of the non-negative orthant of $\mathbb{R}^K$. In *ordinal* MDS, the most common case, Δ is defined as the cone of vectors with $0 \leq \delta_1 \leq \dots \leq \delta_K$, together with some normalization requirement to exclude the trivial solution in which both x and δ (and thus stress) are equal to zero.

1.1 Metric fStress

Groenen et al. (1995) propose a generalization of stress called *fStress*. It is defined as

$$\sigma_f(x) := \sum_{k=1}^K w_k (f(\delta_k) - f(d_k(x)))^2, \quad (3)$$

with f a known real-valued function defined on $(0, +\infty)$. We will only consider functions for which

1. f is strictly increasing on $(0, +\infty)$,
2. f is continuously differentiable on $(0, +\infty)$.

In our implementation we prefer functions with one or more of the following properties.

3. $f(0) = 0$,
4. $\lim_{x \rightarrow \infty} f(x) = \infty$,
5. f is positively homogeneous, i.e. $f(\lambda x) = \lambda^\mu f(x)$ for all $\lambda \geq 0$ and for some $\mu > 0$.
6. f is convex.

Note that (5) implies (3) and (4). Power functions x^r with $r > 0$ clearly satisfy the first five required and preferred conditions, and the sixth one if $r \geq 1$. We will also consider the example of the logarithm (Ramsay (1977)), which does not satisfy preferred conditions (3),(5) and (6). An example of an f that does not satisfy (4), (5) and (6) is the arctangent, bounded by $\pi/2$ as $x \rightarrow \infty$. The logarithm, the arctangent, and the power function with $r \leq 1$ are all concave.

Groenen et al. (1995) show that at a local minimum of *fStress* we have $d_k(x) > 0$ for all k for which $w_k \delta_k > 0$, generalizing the result for stress proved by De Leeuw (1984). Thus, under weak conditions on the weights and dissimilarities, *fStress* is differentiable at local minima.

There does not seem to be an algorithm to minimize *fStress* for general f . However Groenen et al. (1995) derive formulas for the first and second derivatives, which can of course be used in general-purpose optimization routines.

The important special case of *fStress* in which f is a power function is called *rStress* (also known as *powerStress*). It is

$$\sigma_r(x) := \sum_{k=1}^K w_k (\delta_k^r - d_k^r(x))^2. \quad (4)$$

If $r = 1$ we have Kruskal's stress, if $r = 2$ we have *sstress* used by Takane et al. (1977). Using

$$\log(x) = \lim_{r \rightarrow 0} \frac{x^r - 1}{r} \quad (5)$$

shows that the logarithmic loss function used by Ramsay (1977) is a limiting case of rStress.

There have been various attempts to extend the smacof majorization (or MM) method for MDS (De Leeuw (1977)) to rStress. References and links to various largely unpublished reports are in De Leeuw (2017). The recent smacofx package (Rusch et al. (In Press)) has R code for the `rstressMin()` function implementing the majorization method in De Leeuw et al. (2016).

Minimizing (3) can be interpreted as approximating dissimilarities δ by distances $d(x)$ with a weighted least squares loss function, with f acting as a form of weighting in addition to the weights in w . This can be seen most clearly if the fit is good, i.e. when δ and $d(x)$ are close. We can then approximate fStress from (3) as in

$$\sigma_f(x) \approx \sum_{k=1}^K w_k \{f'(\delta_k)\}^2 (\delta_k - d_k(x))^2. \quad (6)$$

See, for example, Groenen and Van de Velden (2016). Minimizing (6) can be done by standard metric smacof. In the case of a close fit this will presumably give a good approximation to minimizing fStress.

1.2 Non-metric fStress

In non-metric MDS we minimize fStress not only over x but also over transformat , where δ is constrained to be in some subset Δ of $\mathbb{R}^K$. To distinguish between dissimilarities and transformed dissimilarities, we redefine fStress as

$$\sigma_f(x, \hat{d}) := \sum_{k=1}^K w_k (\hat{d}_k - f(d_k(x)))^2, \quad (7)$$

and we require that $\hat{d} \in \Delta$. In the MDS literature the $\hat{d}_k$ are called the *disparities* or the *pseudo-distances*. Note that it does not make a difference if we require the disparities $\hat{d}$ to be monotone with the dissimilarities δ or with the transformed dissimilarities $f(\delta)$.

It is important to observe that for rStress we have homogeneity

2 Algorithm

The technique proposed in this paper to minimize non-metric fStress is in the Alternating Least Squares (ALS) family. We start with an initial estimate $x^{(0)}$. We then minimize σ_f over δ in $\Delta \cap \Sigma$ for the current $d(x^{(0)})$. The minimizer $\delta^{(0)}$ is then used to minimize fStress over x for the current disparities, yielding $x^{(1)}$. These two steps are alternated until x and δ do not change any more. Starting in iteration $\nu = 0$ we compute

$$\delta^{(\nu)} = \operatorname{argmin}_{\delta \in \Delta \cap \Sigma} \sigma_f(x^{(\nu)}, \delta), \quad (8a)$$

$$x^{(\nu+1)} = \operatorname{argmin}_x \sigma_f(x, \delta^{(\nu)}). \quad (8b)$$

We then increase ν by one and go into the next iteration. And so on, until convergence.

2.1 Normalized Cone Regression

In metric MDS there is no need for the first step (8a), because $\delta^{(\nu)}$ is equal to the normalized dissimilarities δ for all ν . In the non-metric case step (8a) is needed, but compared to (8b) it is comparatively easy. We have to compute the least squares projection of $f(d(x))$ on the cone Δ and then normalize this projection to give it length one. If Δ is the cone of monotone vectors, as in ordinal MDS, the cone projection is *monotone regression*. The fact that projection on the intersection of Δ and Σ is the same thing as normalizing the projection on Δ is due to De Leeuw (1975) and more generally (and more rigorously) to Bauschke et al. (2018). Since this step is the same as in standard MDS algorithms such as smacof (De Leeuw and Mair (2009), Mair et al. (2022)) we do not go into details here, and just refer to the literature.

2.2 Majorization

The second step (8b), minimizing over x for fixed current δ , is more complicated. There is no analytic solution, unlike the first step, and minimization requires an iterative process of its own. Thus, except in some very special cases, the second step requires an infinite number of “inner” iterations. Since implementing an infinite number of iterations is impossible we have to truncate the inner iteration sequence at some point. In our software we deviate from the strict ALS framework by not minimizing fStress over x for fixed δ , but by taking a single “inner” iteration and merely decrease fStress. If this is done judiciously we still obtain a convergent sequence of updates. This is similar to the strategy in other MDS algorithms such as smacof (De Leeuw (1977)) and alscl (Takane et al. (1977)).

In smacof the inner iteration step is a majorization step, by now more commonly known as an MM step. Briefly, we find a function $\kappa_f(x; y)$ such that

- $\kappa_f(x, y) \leq \sigma_f(x)$ for all x and y in Ξ , and
- $\kappa_f(x; y) = \sigma_f(x)$ if and only if $x = y$.

An inner iteration is of the form

$$x^{(\mu+1)} = \underset{x \in \Xi}{\operatorname{argmin}} \kappa(x, x^{(\mu)}). \quad (9)$$

If $x^{(\mu+1)} = x^{(\mu)}$ we declare convergence and stop. From (9),

$$\sigma_f(x^{(\mu+1)}) \leq \kappa_f(x^{(\mu+1)}, x^{(\mu)}) < \kappa_f(x^{(\mu)}, x^{(\mu)}) = \sigma_f(x^{(\mu)}). \quad (10)$$

Thus an inner majorization step upgrading x for given δ decreases fStress (unless we stop). Remember that in ALS the inner iterative process (indexed by μ) goes on within step 2 of the “outer” update (indexed by ν).

In De Leeuw et al. (2016) a majorization method was proposed to minimize rStress. It majorizes d^r as a function of x , and for this it uses the specific properties of the power function. It consequently cannot be used for fStress. The technique is incorporated in the smacofx R package, and numerical experience indicates it works, but convergence can be painfully slow. Because of the generality of the function f (any differentiable increasing function) it seems impossible to develop an majorization method for fStress along the same lines as the one for rStress. Consequently we go a somewhat different route in this paper. We do not only deviate from the strict ALS procedure by only partially minimizing loss over x for fixed d , we also deviate from the strict majorization approach by majorizing an approximation of fStress.

In deriving the approximation and majorization we suppress the dependency of fStress on δ , because in the second ALS substep δ is just a vector of constants. We first approximate $f(d(x))$ near $d(y)$ with

$$f(d_k(x)) \approx f(d_k(y)) + f'(d_k(y))(d_k(x) - d_k(y)). \quad (11)$$

Define

$$\omega_f(x; y) := \sum_{k=1}^K w_k (f(d_k(x)) - f(d_k(y)) - f'(d_k(y))(d_k(x) - d_k(y)))^2, \quad (12)$$

Note that in (11) the derivative is with respect to d , not with respect to x or y . This is a major difference with the majorizations in De Leeuw et al. (2016).

Note that $\omega_f(x; x) = \sigma_f(x)$ for all x and if f is the identity then $\omega_f(x; y) = \sigma_f(x)$ for all x and y . If f is linear with intercept β and slope α then

$$\omega_f(x, y) = \alpha^2 \sum_{k=1}^K w_k (\delta_k - d_k(x))^2 = \alpha^2 \sigma_r(x), \quad (13)$$

with $r = 1$.

We now give an alternative and more convenient expression for $\omega_f(x, y)$ from (12). Define

$$\tilde{w}_k(y) := w_k \{f'(d_k(y))\}^2, \quad (14a)$$

$$\tilde{\delta}_k(y) := \frac{f(\delta_k) - f(d_k(y))}{f'(d_k(y))} + d_k(y). \quad (14b)$$

Then

$$\omega_f(x; y) = \sum_{k=1}^K \tilde{w}_k(y) (\tilde{\delta}_k(y) - d_k(x))^2. \quad (15)$$

We see from (14a) that $\tilde{w}_k(y)$ is positive if $d_k(y) > 0$. Unfortunately the situation for $\tilde{\delta}_k(y)$ is more complicated. From (14b) it follows that

- if f is convex then $\tilde{\delta}_k(y) \geq \delta_k \geq 0$,
- if f is concave then $\tilde{\delta}_k(y) \leq \delta_k$ and $\tilde{\delta}_k(y)$ can be negative,

as well as

- if $\delta_k > d_k(y)$ then $\tilde{\delta}_k(y) > d_k(y) \geq 0$,
- if $\delta_k < d_k(y)$ then $\tilde{\delta}_k(y) < d_k(y)$ and $\tilde{\delta}_k(y)$ can be negative.

If $\delta_k \approx d_k(y)$ then $\tilde{\delta}_k(y) \approx \delta_k$. In a more precise form

$$\tilde{\delta}_k(y) = \left\{ \frac{f'(\xi)}{f'(d_k(y))} \right\} \delta_k + \left\{ 1 - \frac{f'(\xi)}{f'(d_k(y))} \right\} d_k(y) \quad (16)$$

for some ξ in the interval between δ_k and $d_k(y)$. Or, for twice-differentiable f ,

$$\tilde{\delta}_k(y) = \delta_k + \frac{1}{2} \frac{f''(d_k(y))}{f'(d_k(y))} (\delta_k - d_k(y))^2 + o((\delta_k - d_k(y))^2). \quad (17)$$

For rStress equations (14a) and (14b) become

$$\tilde{w}_k(y) := r^2 w_k d_k^{2(r-1)}(y), \quad (18a)$$

$$\tilde{\delta}_k(y) := \frac{\delta_k^r + (r-1)d_k^r(y)}{r d_k^{r-1}(y)}. \quad (18b)$$

By convexity $\tilde{\delta}_k(y)$ is non-negative if $r \geq 1$. For $0 < r < 1$ we have $\tilde{\delta}_k(y) > 0$ if and only if $d_k(y)/\delta_k < (1-r)^r$.

By writing $\omega_f(x, y)$ as in (15) we are on familiar majorization terrain. If $\delta_k(y) > 0$ we use Cauchy-Schwartz, as in standard smacof,

$$d_k(x) \geq \frac{1}{d_k(y)} \text{tr } x' A_k y, \quad (19a)$$

and if $\delta_k(y) < 0$ we use the arithmetic mean/geometric mean (AM/GM) inequality in the form

$$d_k(x) \leq \frac{1}{2} \frac{1}{d_k(y)} \{y' A_k y + x' A_k x\}, \quad (19b)$$

Use of the AM/GM inequality for majorizing terms with negative dissimilarities was pioneered by Heiser (1991).

Define the matrices

$$V(y) := \sum_{k=1}^K \tilde{w}_k(y) A_k, \quad (20a)$$

$$B(y) := \sum_{\tilde{\delta}_k(y) > 0} \tilde{w}_k \frac{\tilde{\delta}_k(y)}{d_k(y)} A_k, \quad (20b)$$

$$H(y) := \sum_{\tilde{\delta}_k(y) < 0} \tilde{w}_k \frac{|\tilde{\delta}_k(y)|}{d_k(y)} A_k. \quad (20c)$$

Note that all three matrices (more precisely matrix-valued functions) are positive semi-definite. Now, using these matrices and the inequalities (19a) and (19b),

$$\omega_f(x, y) \leq \kappa(x, y) := \gamma(y) + x'(V(y) + H(y))x - 2x'B(y)y, \quad (21)$$

with

$$\gamma(y) := \sum_{k=1}^K \tilde{w}_k(y) \tilde{\delta}_k^2(y) + y'H(y)y \quad (22)$$

which does not depend on x .

We now update using

$$x^{(\mu+1)} = \underset{x}{\operatorname{argmin}} \kappa_f(x; x^{(\mu)}) = (V(x^{(\mu)}) + H(x^{(\mu)}))^+ B(x^{(\mu)}) x^{(\mu)},$$

which results in

$$\sigma_f(x^{(\mu+1)}) \approx \omega_f(x^{(\mu+1)}; x^{(\mu)}) \leq \kappa_f(x^{(\mu+1)}; x^{(\mu)}) < \kappa_f(x^{(\mu)}; x^{(\mu)}) = \sigma_f(x^{(\mu)}).$$

Unlike in smacof there is no guarantee that σ_f decreases from one iteration to the other. Monotone convergence of loss function values will only happen if the approximation is good, i.e. if $d(x^{(\mu+1)})$ is close to $d_k(x^{(\mu)})$. This requires closer monitoring of convergence than in smacof. Also note that if $\tilde{\delta}_k(y) < 0$ for all k then $B(y) = 0$, which means that the update from y is equal to zero. Again, this eventuality must be monitored.

3 Software

The R function `smacofSSRStressR()` and the C function `smacofSSRStressC()` in the files `smacofSSFStressR.R` and `smacofSSFStressC.c` handle both metric and ordinal MDS minimizing f Stress, in principle for an arbitrary f . If we refer to `smacofSSRStressX()` we mean both the R and the C function.

The R code uses the `gpava()` function from De Leeuw et al. (2009) for monotone regression, the C code uses `monotone()` from Busing (2022).

In C the Moore-Penrose inverse of a symmetric doubly-centered matrix V is computed with the `syminv()` routine from Healy (1968), using the C translation by John Burkardt at https://people.sc.fsu.edu/~jburkardt/c_src/c_src.html. In both cases we use the formula

$$V^+ = (V + n^{-1}ee')^{-1} - n^{-1}ee'. \quad (23)$$

This gives the correct result if the rank of V is $n - 1$, which happens if and only if the weights are irreducible. This means that in some (rare) cases we may run into a singular V .

The C code is compiled into a shared library `smacofSSFStressC.so`, which is loaded by the R front-end `smacofSSFStressC.R`.

3.1 Data

For `smacofSSRStressR.R` and `smacofSSRStressC.R` the data and weights are in the MDS data format defined in De Leeuw (2025). This is a list with two scalars `nobj` and `ndat`, the number of objects and the number of data points (object pairs), and five vectors `iind`, `jind`, `delta`, `blocks`, `weights` of length `ndat`. Vectors `iind` and `jind` have integers between 1 and `n` with `iind > jind` that code for which pairs of objects we have dissimilarity information. The dissimilarities are in `delta`, the weights in `weights`. Dissimilarities are sorted in increasing order. The `blocks` vector codes the tie-blocks in the dissimilarities.

3.2 Initial Estimate

There are two possibilities. If `xinit` is not `NULL` then it is used as the initial estimate of the configuration. This is useful, for example, if we start the iterations for one f with the solution for another. If `xinit` is `NULL` then the function `smacofSSRStressInit()` computes an initial estimate using classical MDS (also known as Torgerson-Gower MDS) on the dissimilarities.

First we make sure the weights add up to one. If there are missing dissimilarities they are first imputed by setting them equal to the weighted mean of the non-missing dissimilarities. Then

we double center the squared δ and compute the classical scaling solution using the `eigs_sym()` function from the `RSpectra` package (Qiu and Mei (2024)).

It is well-known that if the fit is half-way decent then the distances in the Torgerson-Gower solution underestimate the dissimilarities. It makes sense, consequently, to multiply the initial configuration by a constant to get a better fit (Malone et al. (2002)). We use the `fStress` approximation (6) and minimize

$$\sigma(\lambda) = \sum_{k=1}^K w_k \{f'(\delta)\}^2 (\delta_k - d_k(\lambda x))^2$$

over λ .

3.3 Functions

The R and C versions come with the following hard-coded functions. The *what* (zero to four) selects the function, the *rpow* parameter gives the *r* parameter value.

Table 1: f examples

	$f(x)$	$f'(x)$	$f^{-1}(y)$
power	x^r	rx^{r-1}	$y^{1/r}$
shifted log	$\log(rx + 1)$	$r/(rx+1)$	$(\exp(y) - 1) / r$
shifted exponent	$\exp(rx) - 1$	$r \exp(rx)$	$\log(y + 1)/r$
log	$r \log(x)$	r/x	$\exp(y/r)$
atan	$\text{atan}(rx)$	$r / (1 + (rx)^2)$	$\tan(y)/r$

The selection code is in the C function `smacofSSFStressFlist()` and in the R function `smacofSSFStressSelect()`. It is easy to add functions to both versions, but adding them to the C version requires recompiling of the shared library.

3.4 Arguments

The arguments of `smacofSSFStressR()` and `smacofSSFStressR()` are

Parameter *r* is included in case *f* and *g* need additional arguments (as in the case of `fPower` and `gPower`). Since all code is in R, it is easy to add additional transformation functions if needed.

3.5 Value

The result of `smacofSSRStressX()` is

```
result <- list(  
  delt = theData$delta,  
  dhat = h$dhat,  
  dist = h$dnew,  
  xmat = matrix(h$xnew, nobj, ndim),  
  wmat = h$wght,  
  loss = h$snew,  
  ndim = ndim,  
  init = xinit,  
  itel = h$itel,  
  nobj = nobj,  
  iind = h$iind,  
  jind = h$jind,  
  ordi = ordinal,  
  ties = h$ties,  
  what = what,  
  rpow = rpow,  
  labl = labl,  
  date = date()  
)
```

The input parameters are copied in the output so that the result has all the relevant information to reconstruct the run, and can pass it, for example, to the plot routines.

3.6 Plots

The four standard plots that come with the recent `smacof` programs () are

- `smacofConfigurationPlot()`
- `smacofShepardPlot()`
- `smacofDistDhatPlot()`
- `smacofResidualPlot()`

The output list of `smacofSSFStress()` can be fed immediately into these four plot routines. They only have side effects (i.e. make a plot) and return NULL.

3.6.1 smacofConfigurationPlot()

It is easy to explain what `smacofConfigurationPlot()` does. It plots a two-dimensional projection of the configuration. Its arguments are self-explanatory.

```
function(h, labels = NULL, dim1 = 1,
         dim2 = 2, pch = 16, col = "RED", cex = 1)
```

3.6.2 smacofShepardPlot()

A Shepard plot (De Leeuw and Mair (2015)) traditionally plots observed dissimilarities on the horizontal axis, and both disparities and distances on the vertical axis. The (δ_k^0, δ_k) points are connected by a straight line through the origin in the metric case and by an increasing step function in the ordinal case. The (δ_k, d_k) points are optionally connected with vertical “fitlines” to the corresponding $(\delta_k, \hat{d}_k)$ points.

```
function(h, main = "ShepardPlot", fitlines = TRUE, colline = "RED",
         colpoint = "BLUE", type = 0, lwd = 2, cex = 1, pch = 16)
```

In the fStress solution it makes sense to put the final δ and $f(d(x))$ on the vertical axis, but there is some ambiguity on what we use on the horizontal axis. As explained in the introduction in metric MDS we can have $\delta \propto f(\delta^0)$, with δ^0 observed dissimilarities. In that case we put $f(\delta^0)$ on the horizontal axis, and the the points $f(\delta_k^0), \delta_k$ are on a straight line.

In the numerical case do we analyze $\delta = f(\delta^0)$ or $\delta = \delta^0$. In the ordinal case: it does not make a difference, except for the plots

In the fStress context there is an extra arguments “type”. Remember that the list `h` has both a “what” and an “rpow” element. The “type” argument is either one or zero. If it is zero (the default) then we plot $f(\delta_k)$ on the horizontal axis and $\hat{d}_k$ and $f(d_k(x))$ on the vertical axis. If “type” is equal to one then we plot δ_k on the horizontal axis and $f^{-1}(\hat{d}_k)$ and $d_k(x)$ on the vertical axis.

3.6.3 smacofDistDhatPlot()

`smacofDistDhatPlot()` has arguments

```
function(h, fitlines = TRUE, colline = "RED", colpoint = "BLUE",
  main = "Dist-Dhat Plot", type = 0, cex = 1, lwd = 2, pch = 16)
```

The “type” parameter works in the same way as in the Shepard plot.

3.6.4 smacofResidualPlot()

smacofResidualPlot() is more complicated. It has arguments

```
function (h, main = "ResidualPlot", probs = seq(0, 1, 0.25),
  whatres = 1, whatplot = 1, type = 0, qlines = TRUE, colpoints = "RED",
  collines = "BLUE", lwdpoints = 2, lwdlines = 2, cex = 1,
  pch = 16)
```

As in the previous plots *type* computes residuals as

whatplot can have the values *ecdf* (default) or *density*. Computation is done with the `ecdf()` and `density()` functions from the `stats` package.

whatres chooses to plot the residual (default), its absolute value, or its square

4 Example

4.1 Functions

We analyse the colour data from Ekman (1954) with seven different functions f , using both `smacofSSFStressR()` and using `smacofSSFStressC()`, both metric and ordinal.

Table 2: `smacofSSFStressC()` results Ekman Data

	itel	fStress
shifted logarithm	154	0.00418
power .1	804	0.00026
power .5	227	0.00467
power 1	35	0.01160
power 2	49	0.01824
power 5	793	0.00803
shifted exponent	36	0.04511

Table 3: `smacofSSFStressC()` results Ekman Data, ordinal MDS

	itel	fStress
shifted logarithm	336	0.0002466
power .1	980	0.0000066
power .5	213	0.0001651
power 1	141	0.0005337
power 2	231	0.0009015
power 5	2625	0.0011002
shifted exponential	159	0.0007895

4.2 Timings

The package microbenchmark (Mersmann (2024)) can be used to compare the R and C versions. We report the median time for 100 runs of each analysis. In the metric case the C version is between 10 and 30 times faster than the R version, with the smallest difference for the stress ($r = 1$) and sstress ($r = 2$) solutions. In the non-metric case the C version is consistently between 40 and 50 times faster. In packages such as `smacof` and `smacofx` the majorization step

is done in R and the monotone regression step is done in C. In that case the speedup of our all-C version will be somewhere between the metric and non-metric speedsups in the tables.

Table 4: R and C timings Ekman Data in milliseconds, metric MDS

	R times	C times	R/C
shfted logarithm	14.992	0.575	26
power .1	80.865	2.984	27
power .5	22.245	1.040	21
power 1	3.641	0.366	10
power 2	4.947	0.431	11
power 5	79.643	2.900	27
shifted exponential	3.707	0.340	11

Table 5: R and C timings Ekman Data in milliseconds, nonmetric MDS

	R times	C times	R/C
shifted logarithm	113.944	2.819	40
power .1	430.655	9.030	48
power .5	112.219	2.245	50
power 1	58.316	1.466	40
power 2	96.421	2.366	41
power 5	1178.645	23.513	50
shifted exponential	64.072	1.500	43

4.3 Plots

Although we limit ourselves to a single example, the number of plots that can be made is unlimited, because we can choose f in so many ways. Moreover there are four different types of plots for each analysis, and each of the four types has parameters that influence the plot. A true embarrassment of riches.

In order not to go overboard we limit ourselves to ten analysis, minimizing rStress for $r = .1, .5, 1, 2, 5$, both metric and ordinal. For each analysis we give the configuration plot, the Shepard plot, and the EDF of the residuals.

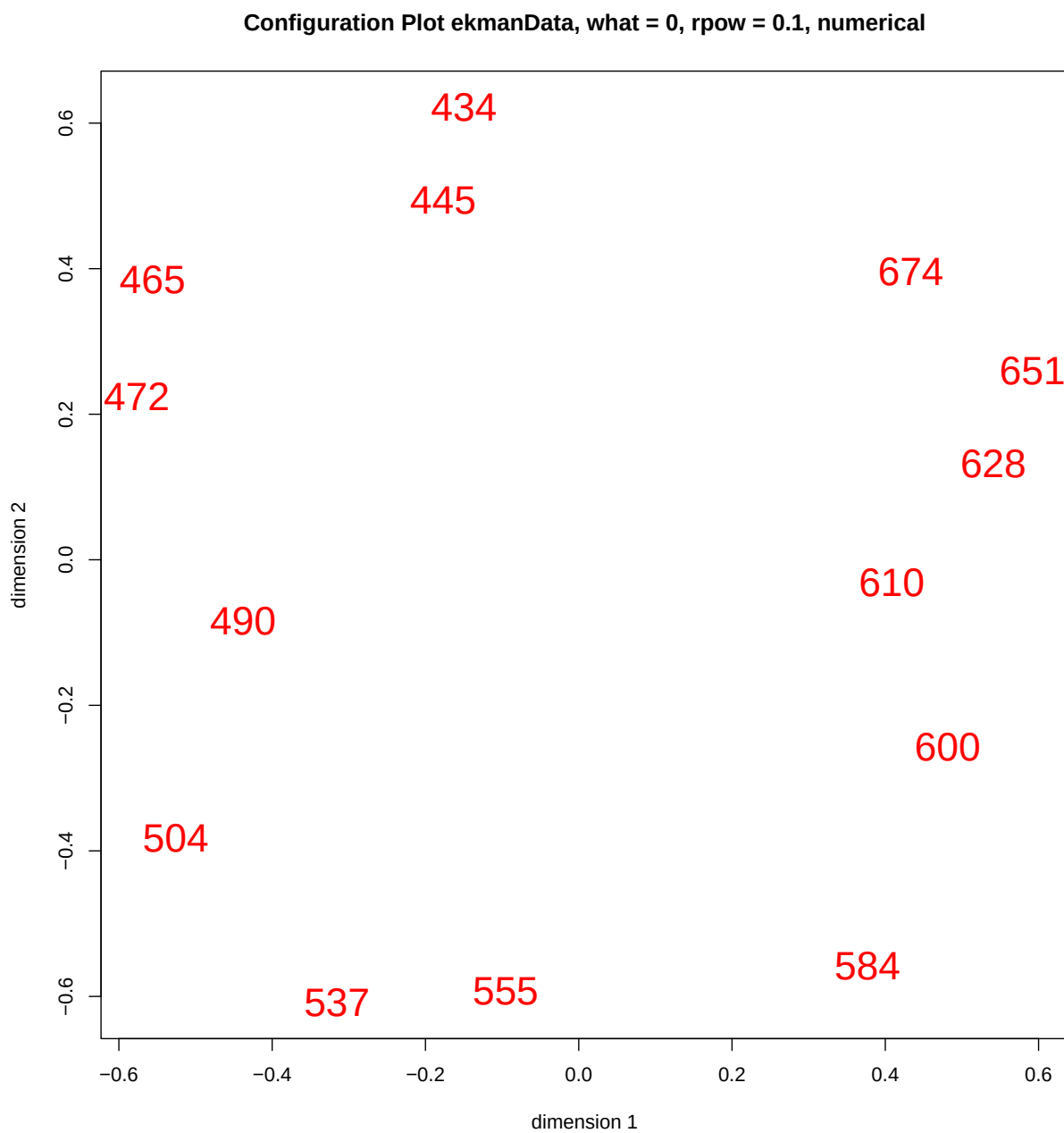


Figure 1: Configuration Plot, Metric, $r = .1$

ShepardPlot ekmanData, what = 0, rpow = 0.1, numerical

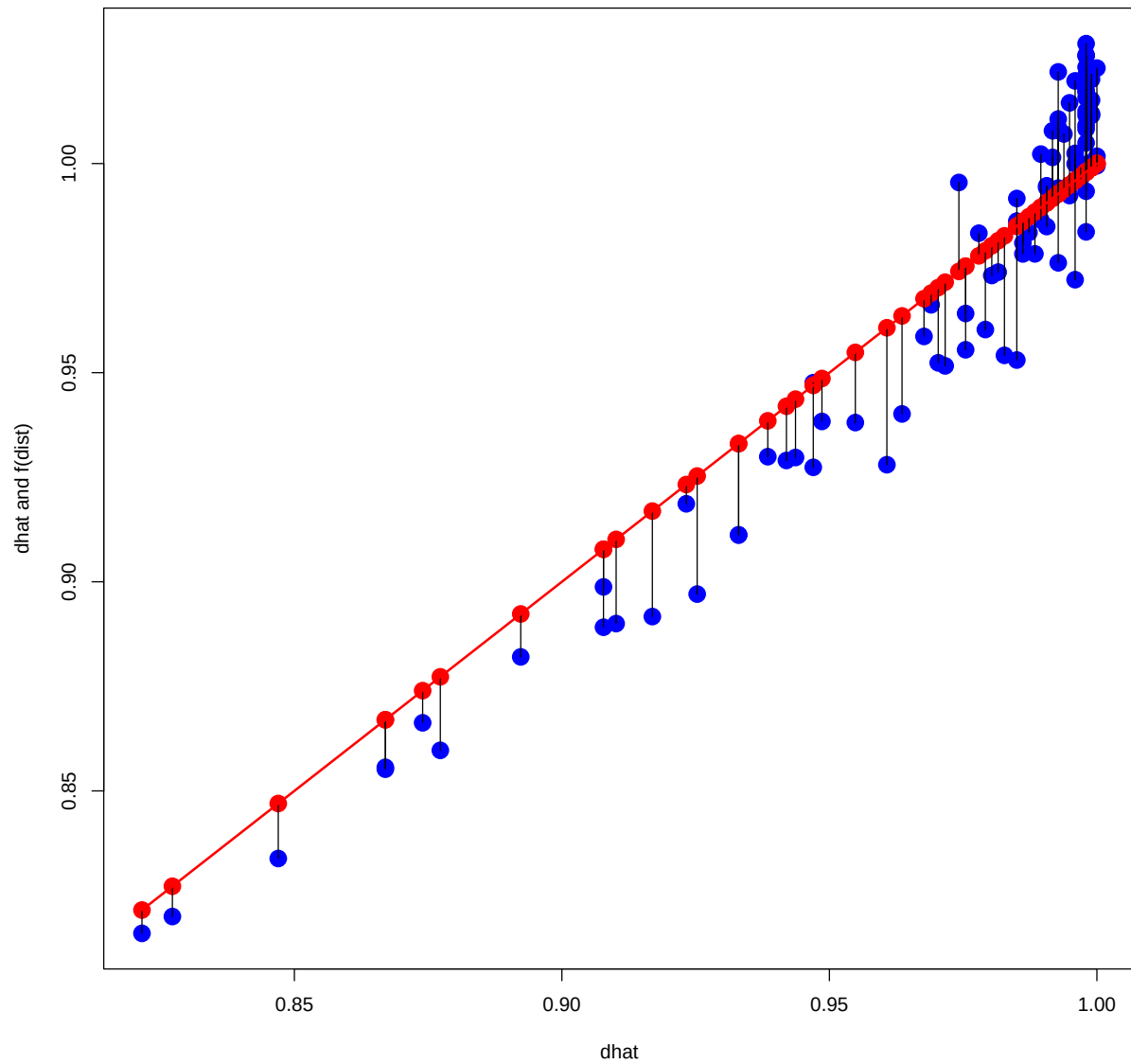


Figure 2: Shepard Plot, Metric, $r = .1$

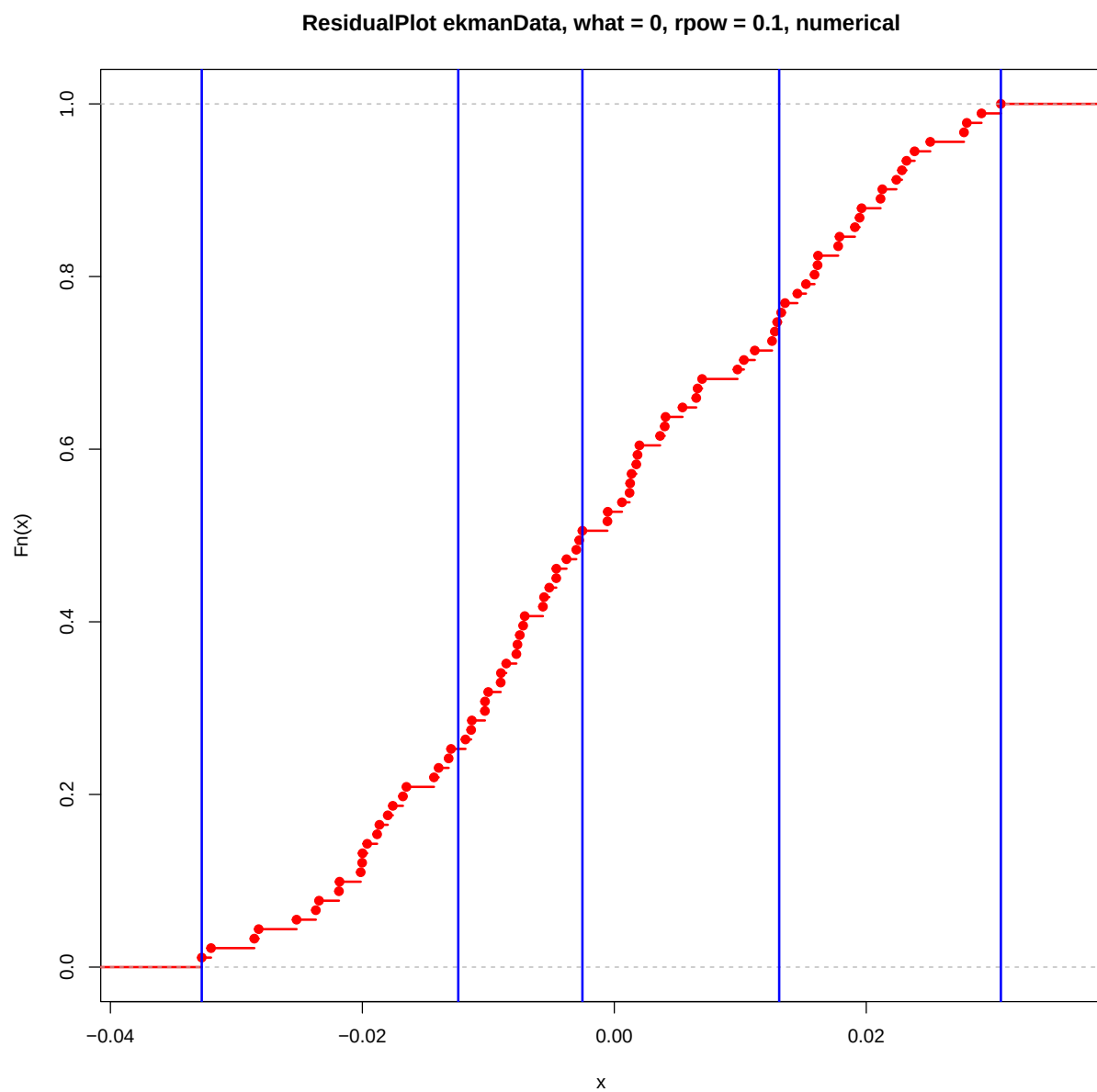


Figure 3: Residual EDF Plot, Metric, $r = .1$

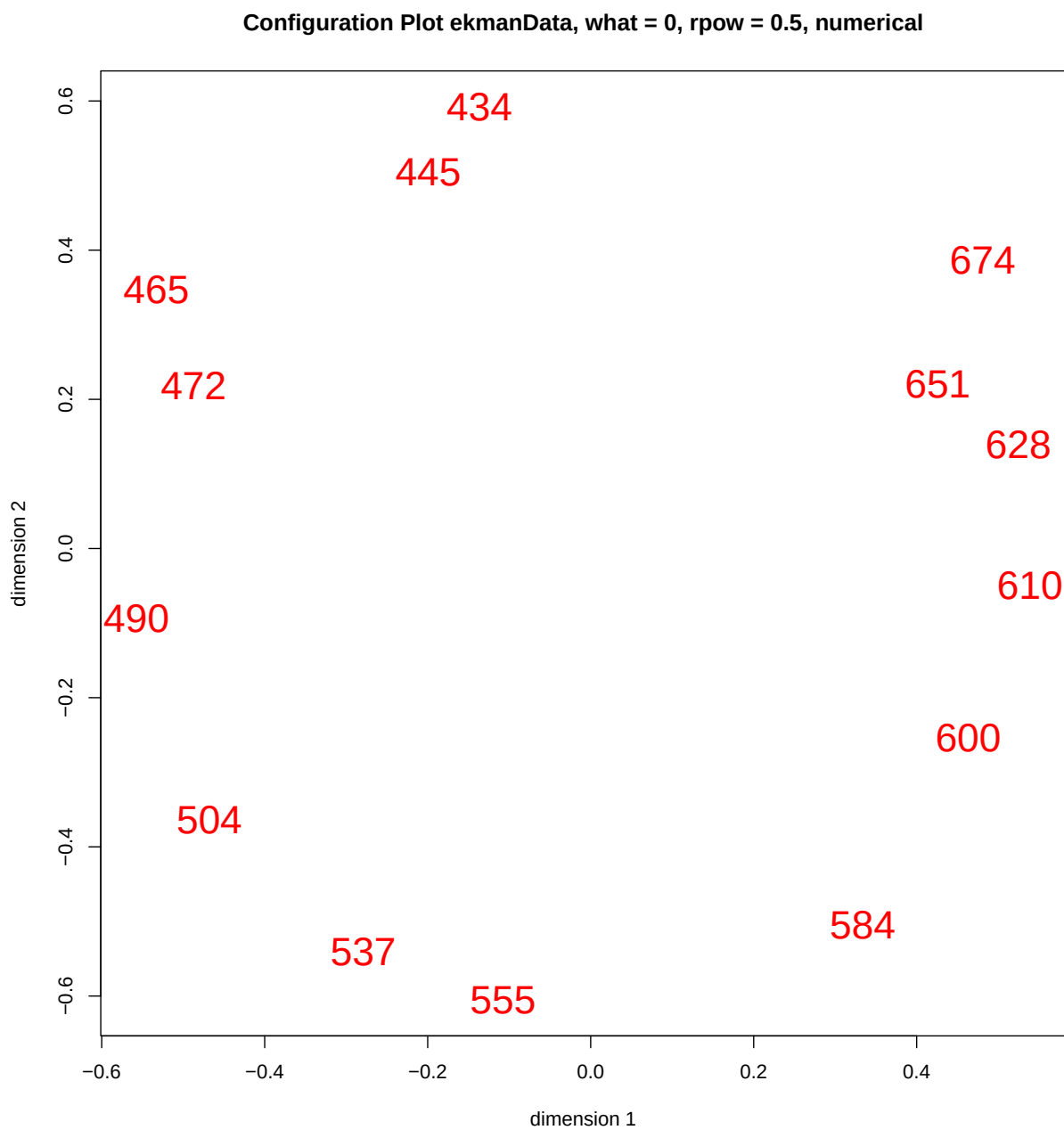


Figure 4: Configuration Plot, Metric, $r = .5$

ShepardPlot ekmanData, what = 0, rpow = 0.5, numerical

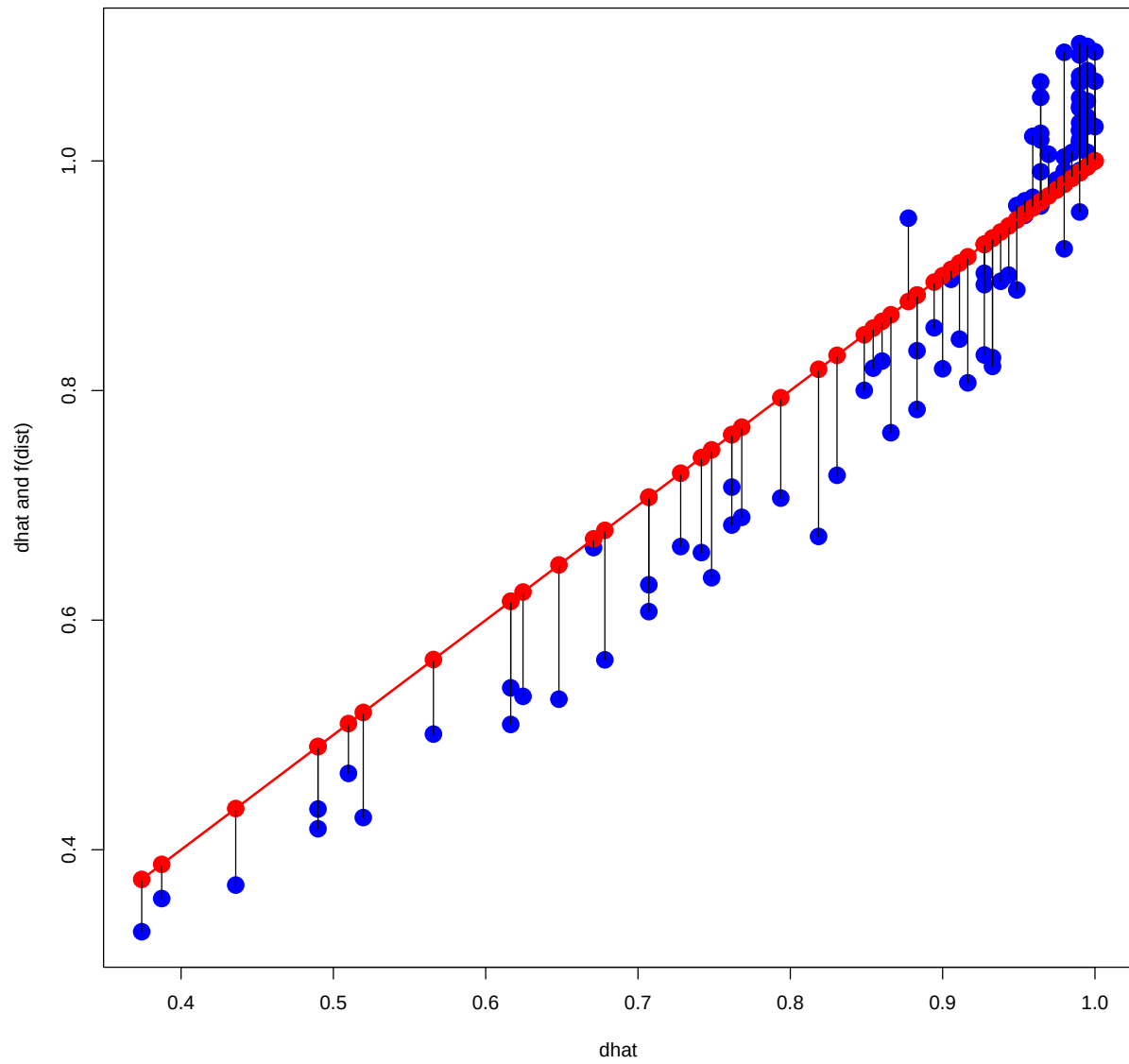


Figure 5: Shepard Plot, Metric, $r = .5$

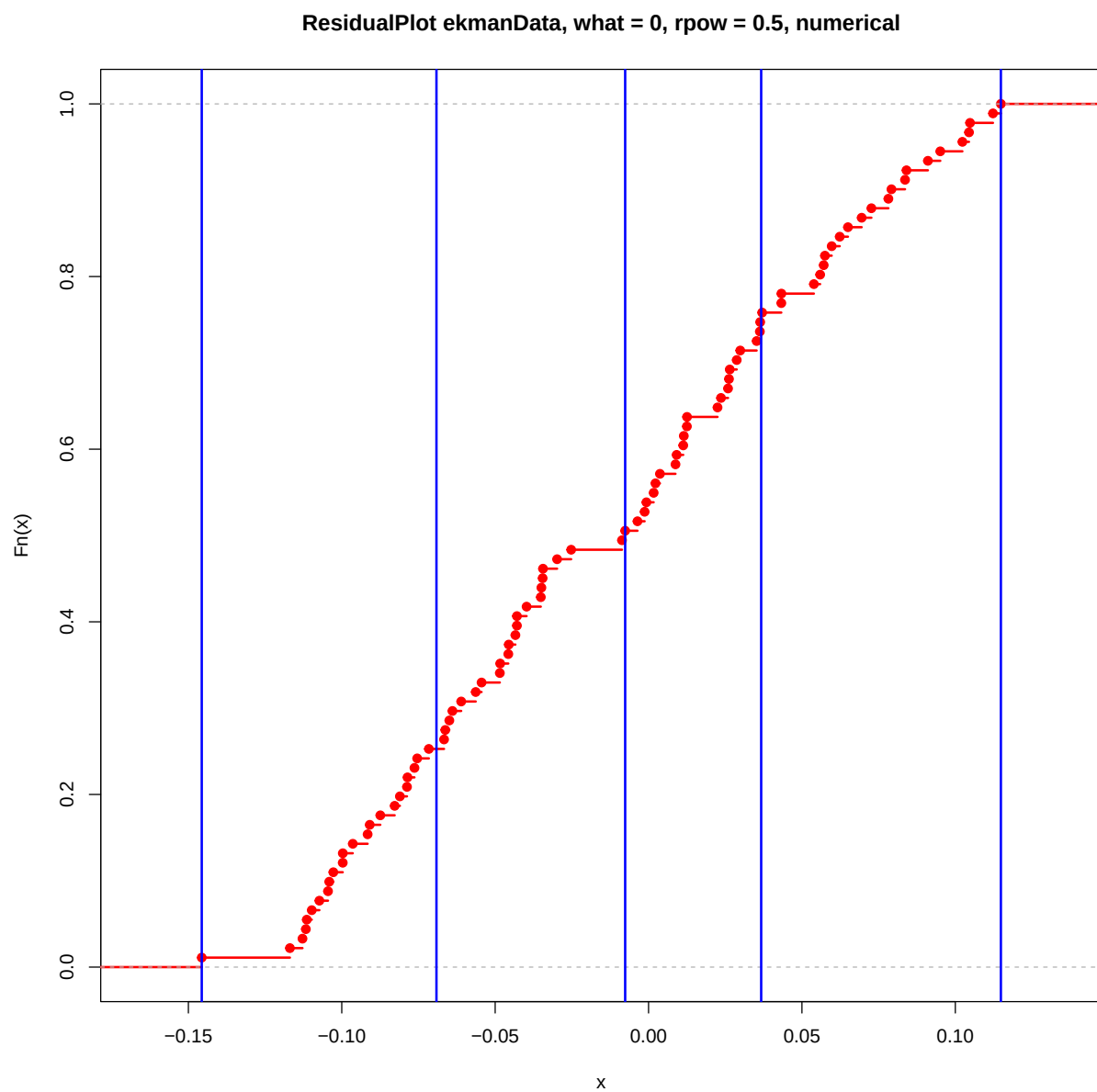


Figure 6: Residual EDF Plot, Metric, $r = .5$

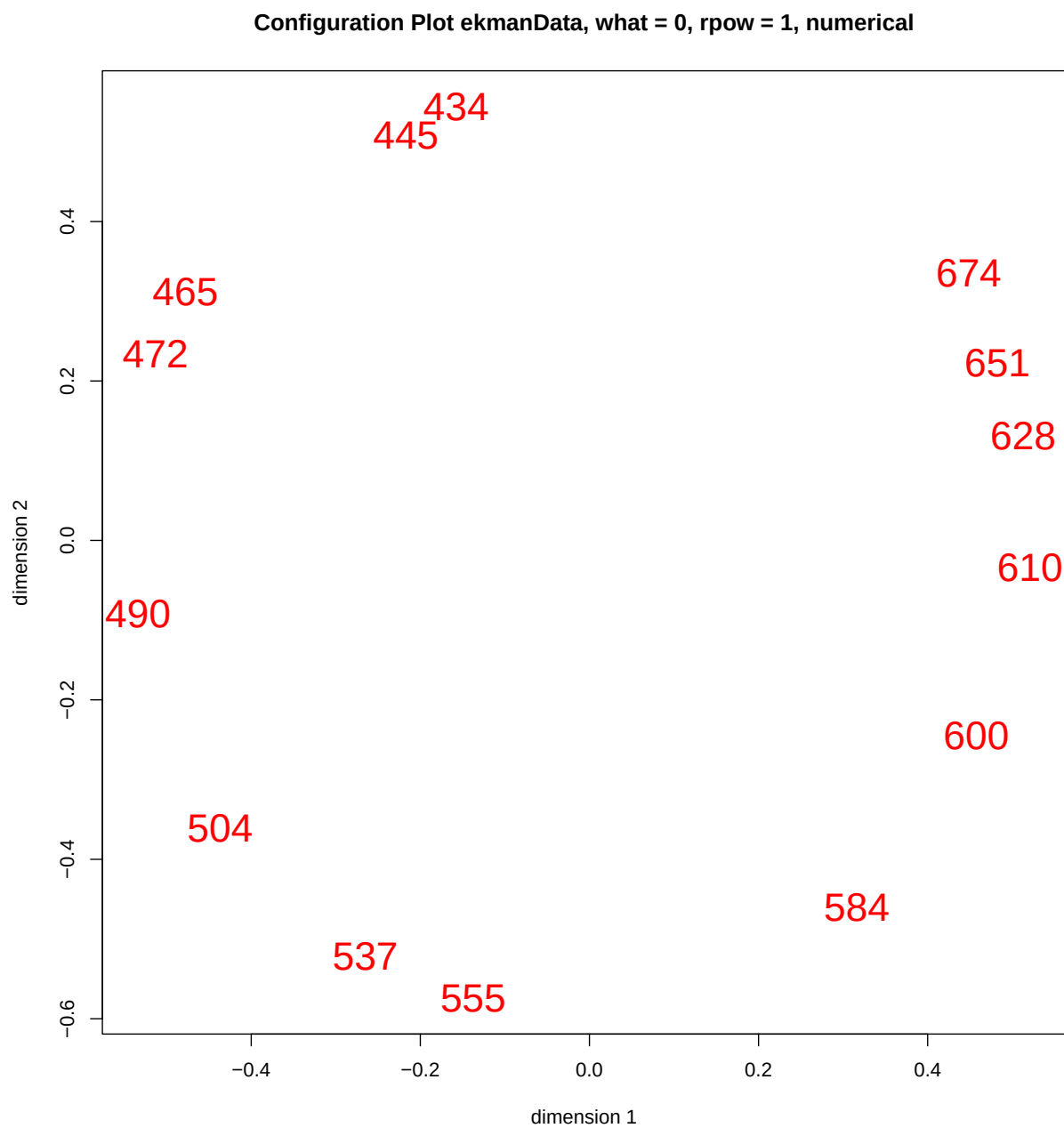


Figure 7: Configuration Plot, Metric, $r = 1$

ShepardPlot ekmanData, what = 0, rpow = 1, numerical

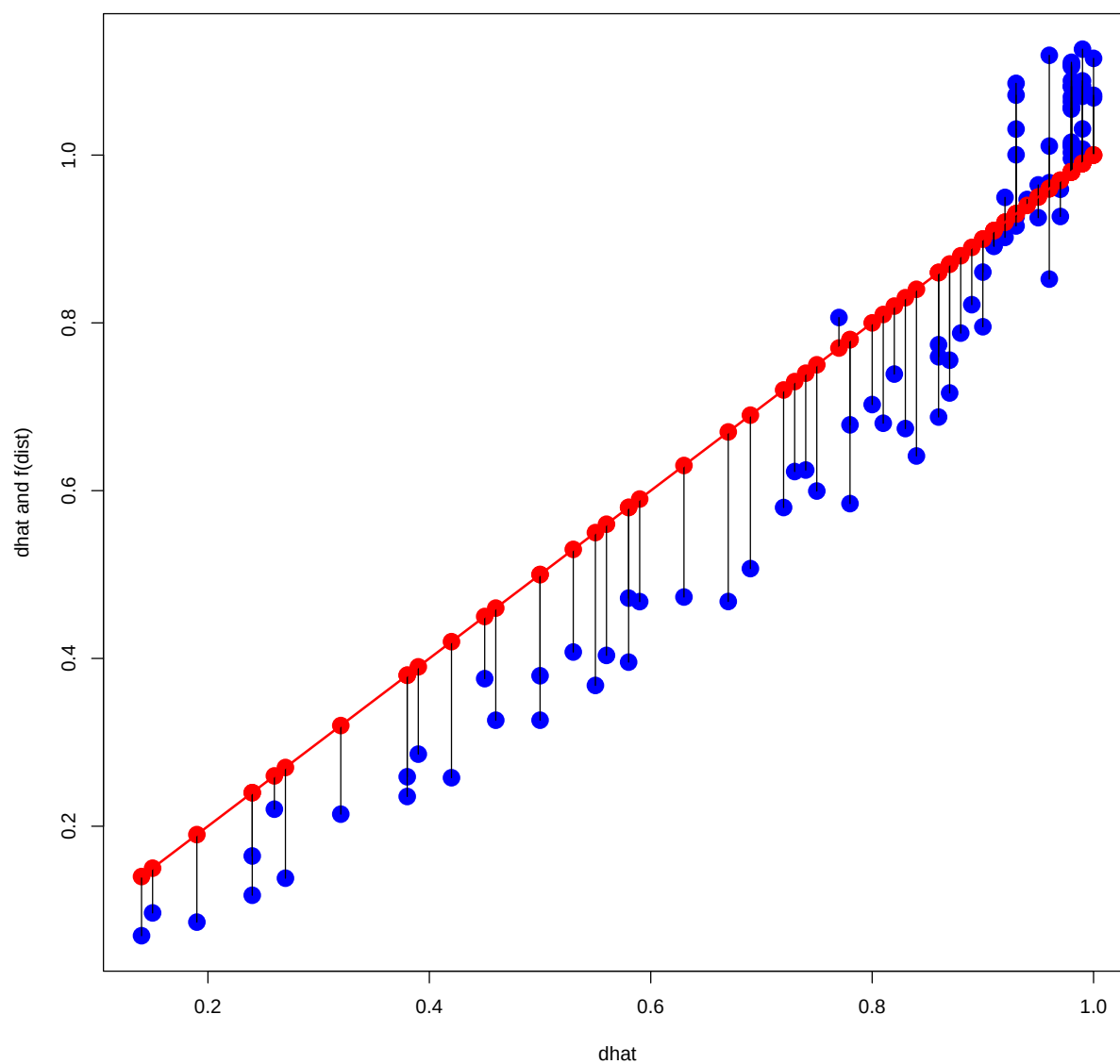


Figure 8: Shepard Plot, Metric, $r = 1$

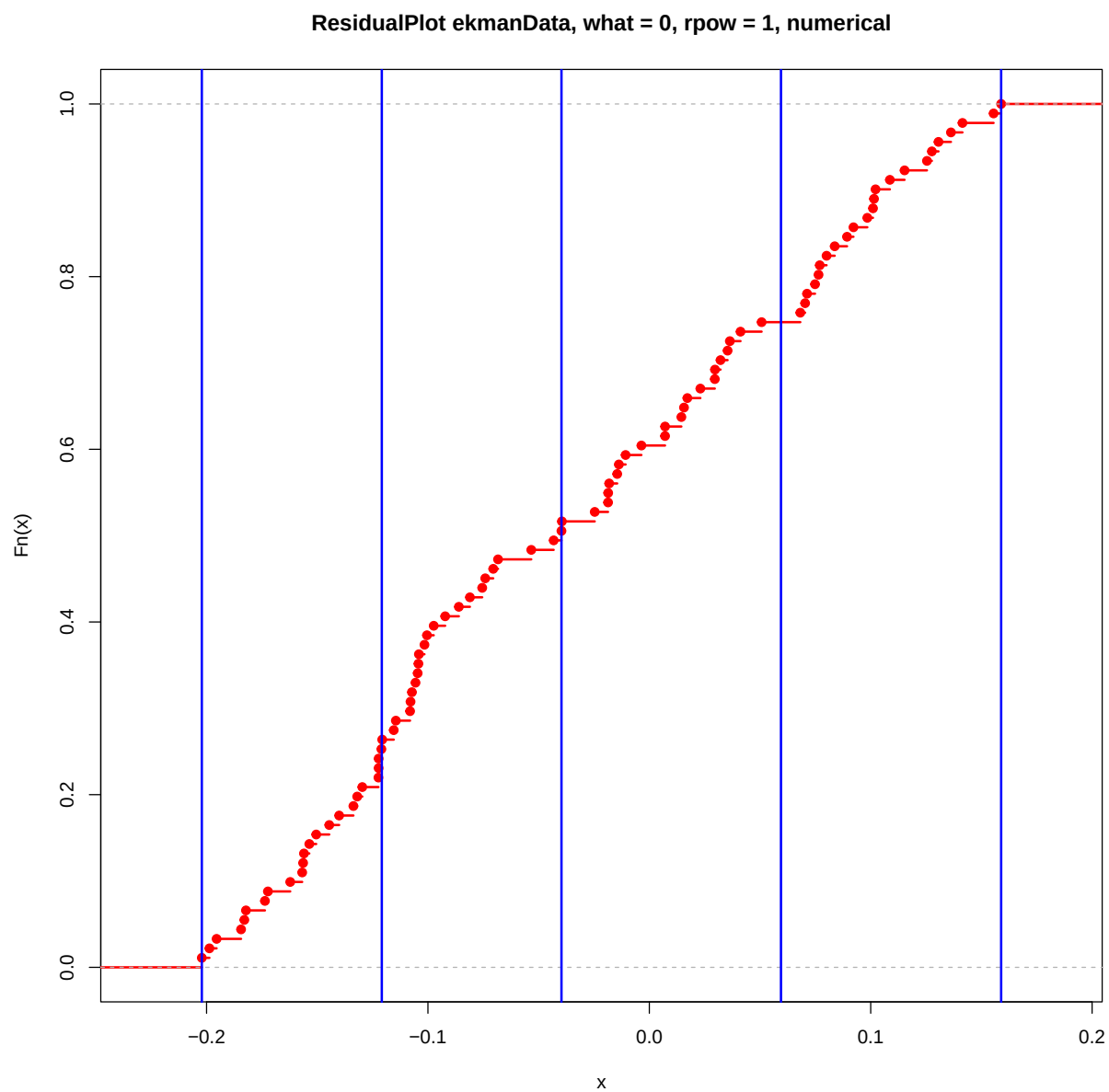


Figure 9: Residual EDF Plot, Metric, $r = 1$

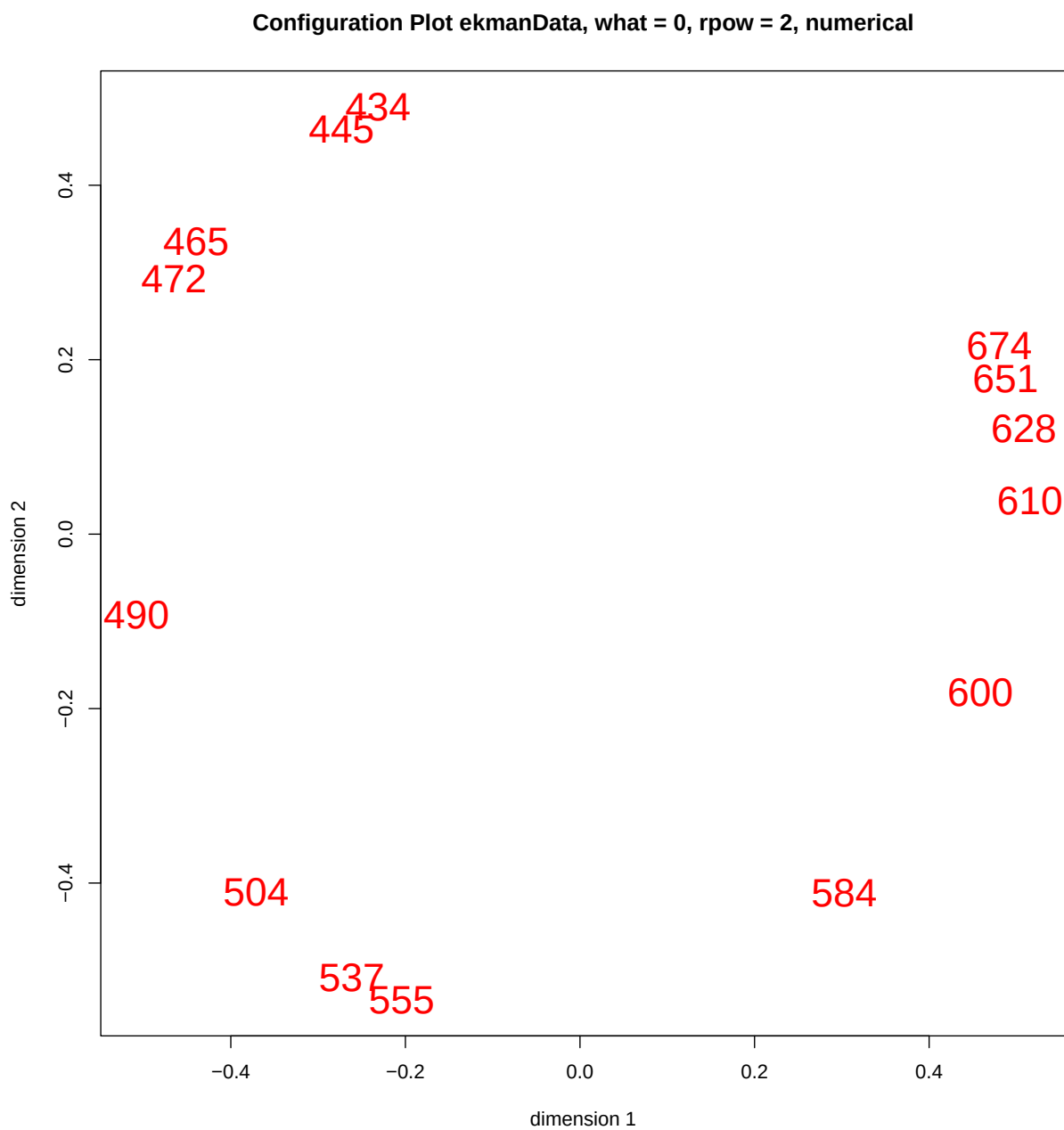


Figure 10: Configuration Plot, Metric, $r = 2$

ShepardPlot ekmanData, what = 0, rpow = 2, numerical

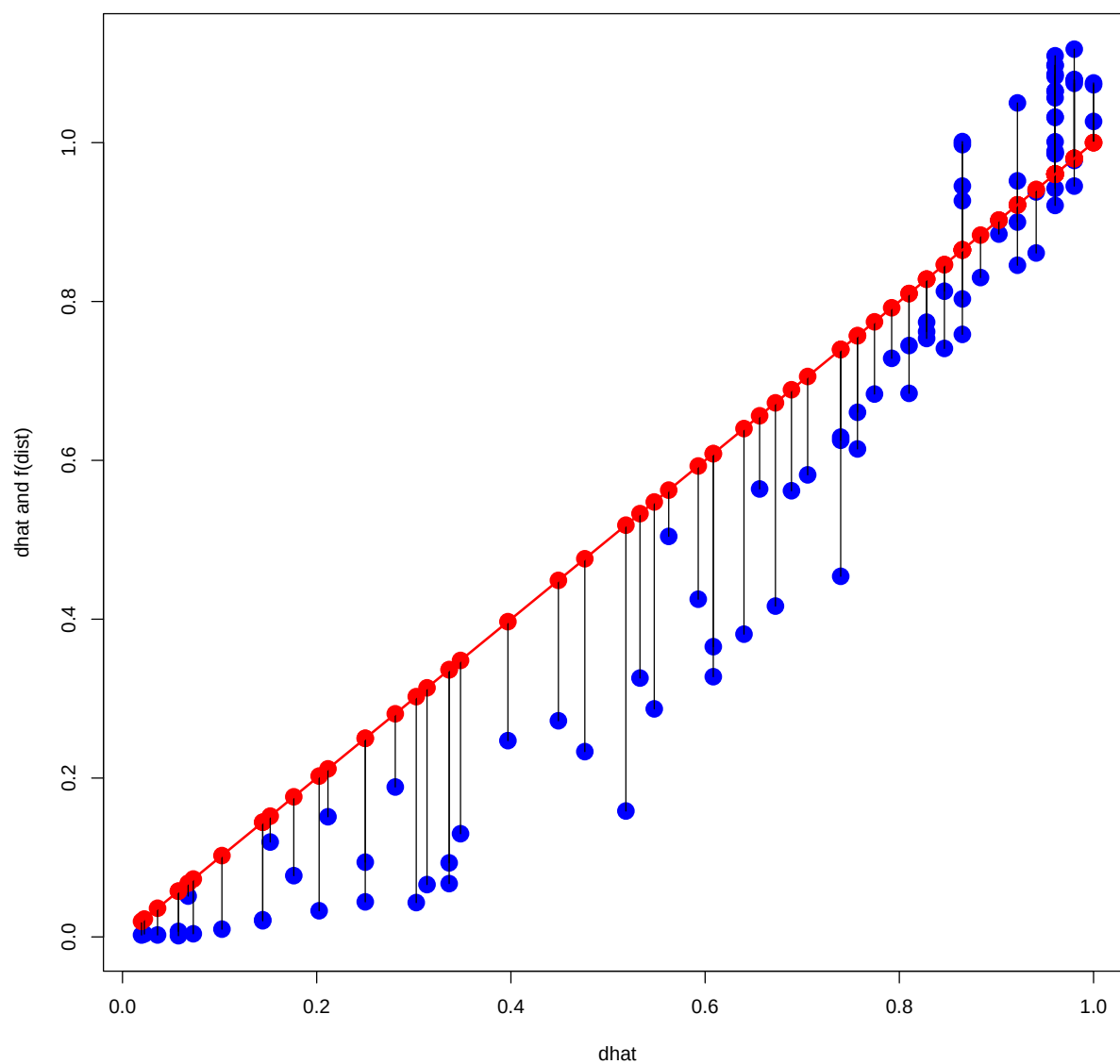


Figure 11: Shepard Plot, Metric, $r = 2$

ResidualPlot ekmanData, what = 0, rpow = 2, numerical

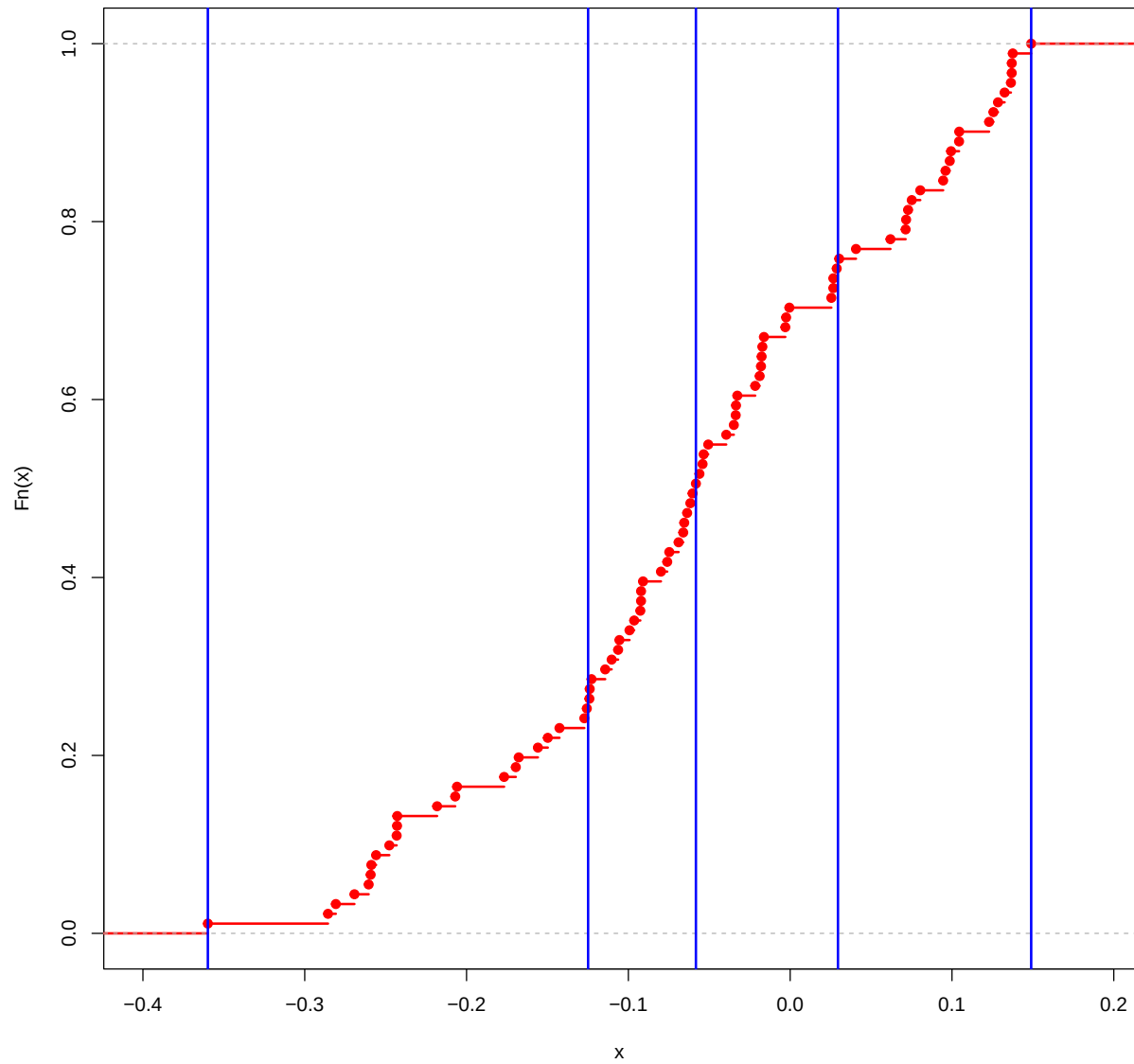


Figure 12: Residual EDF Plot, Metric, $r = 2$

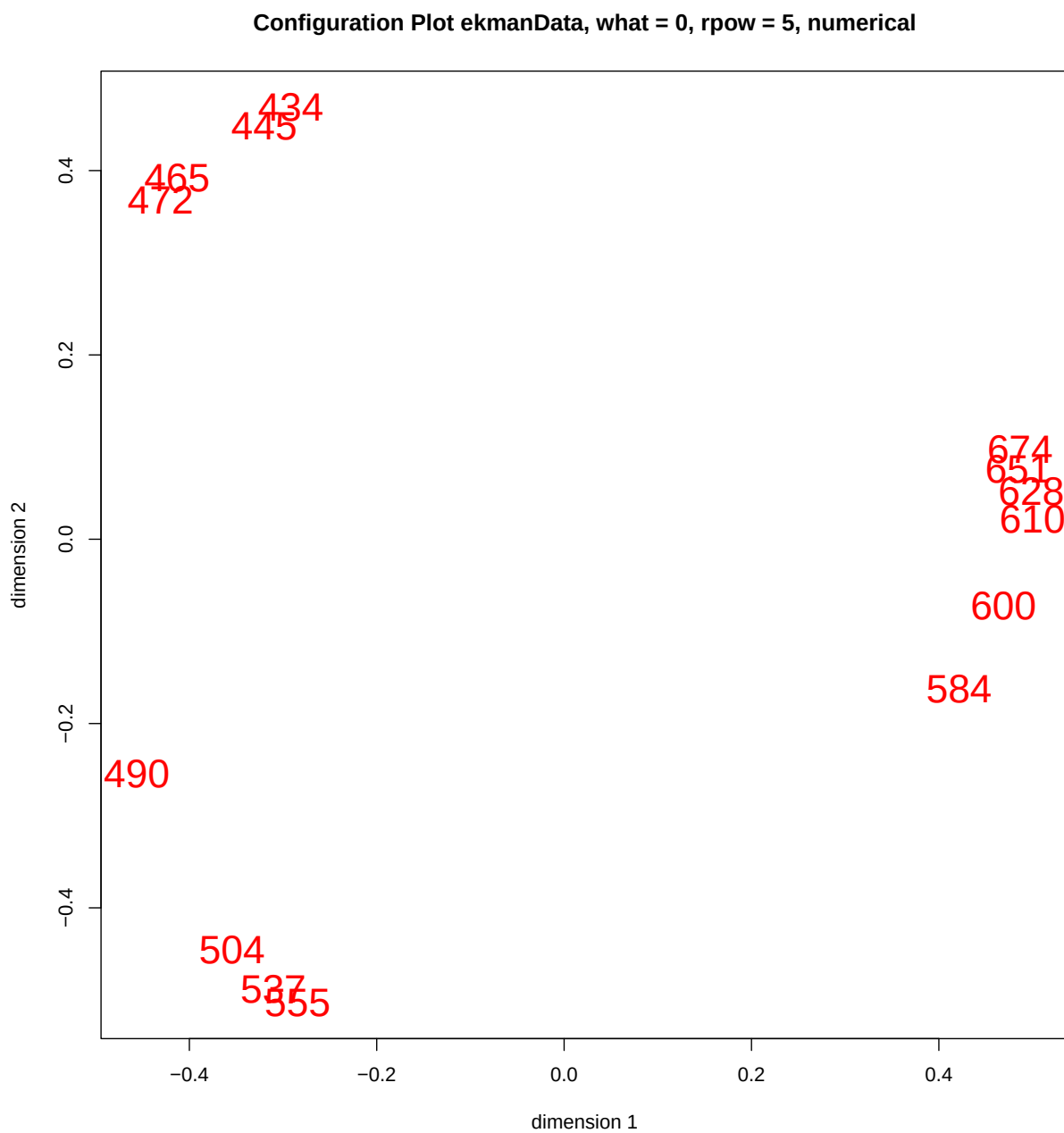


Figure 13: Configuration Plot, Metric, $r = 5$

ShepardPlot ekmanData, what = 0, rpow = 5, numerical

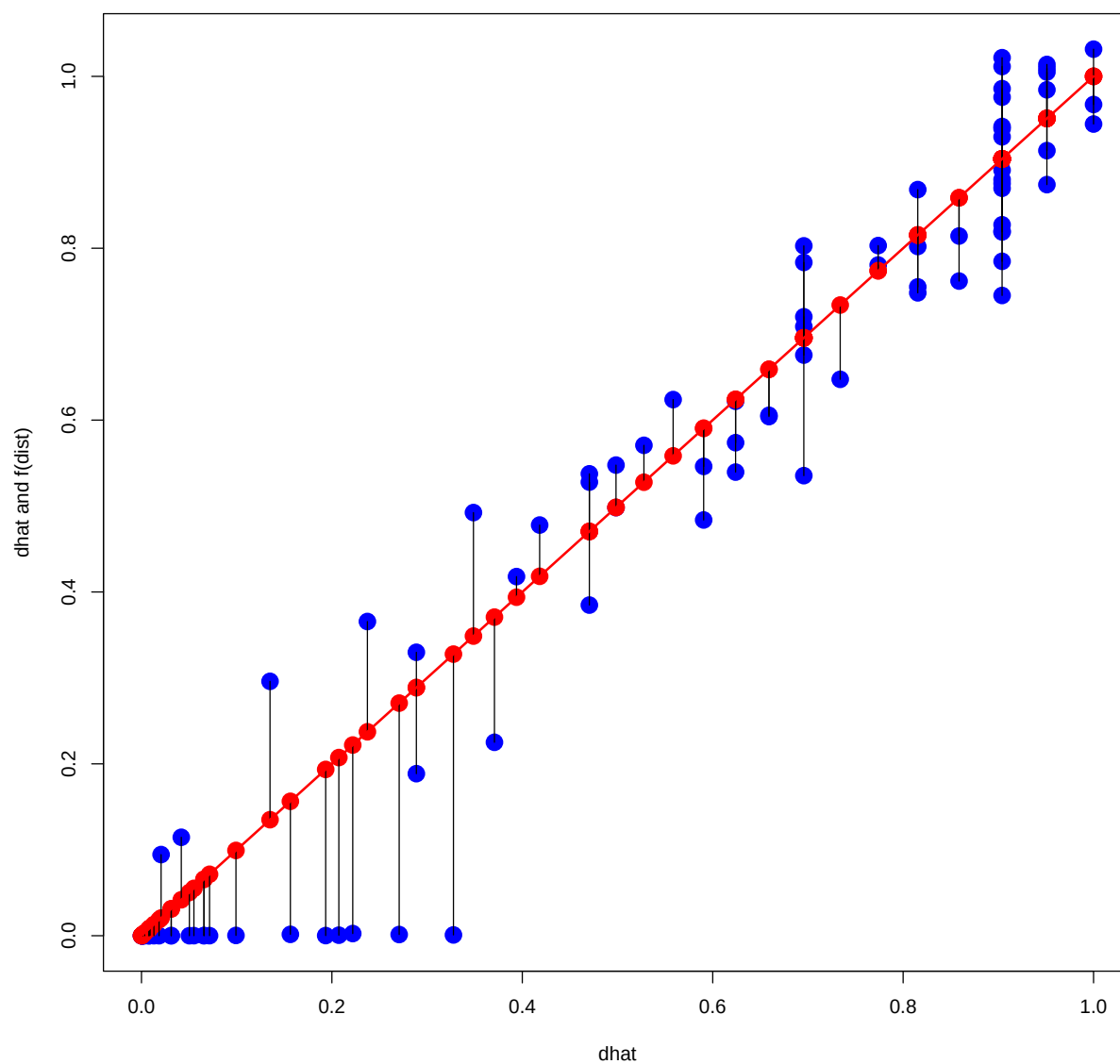


Figure 14: Shepard Plot, Metric, $r = 5$

ResidualPlot ekmanData, what = 0, rpow = 5, numerical

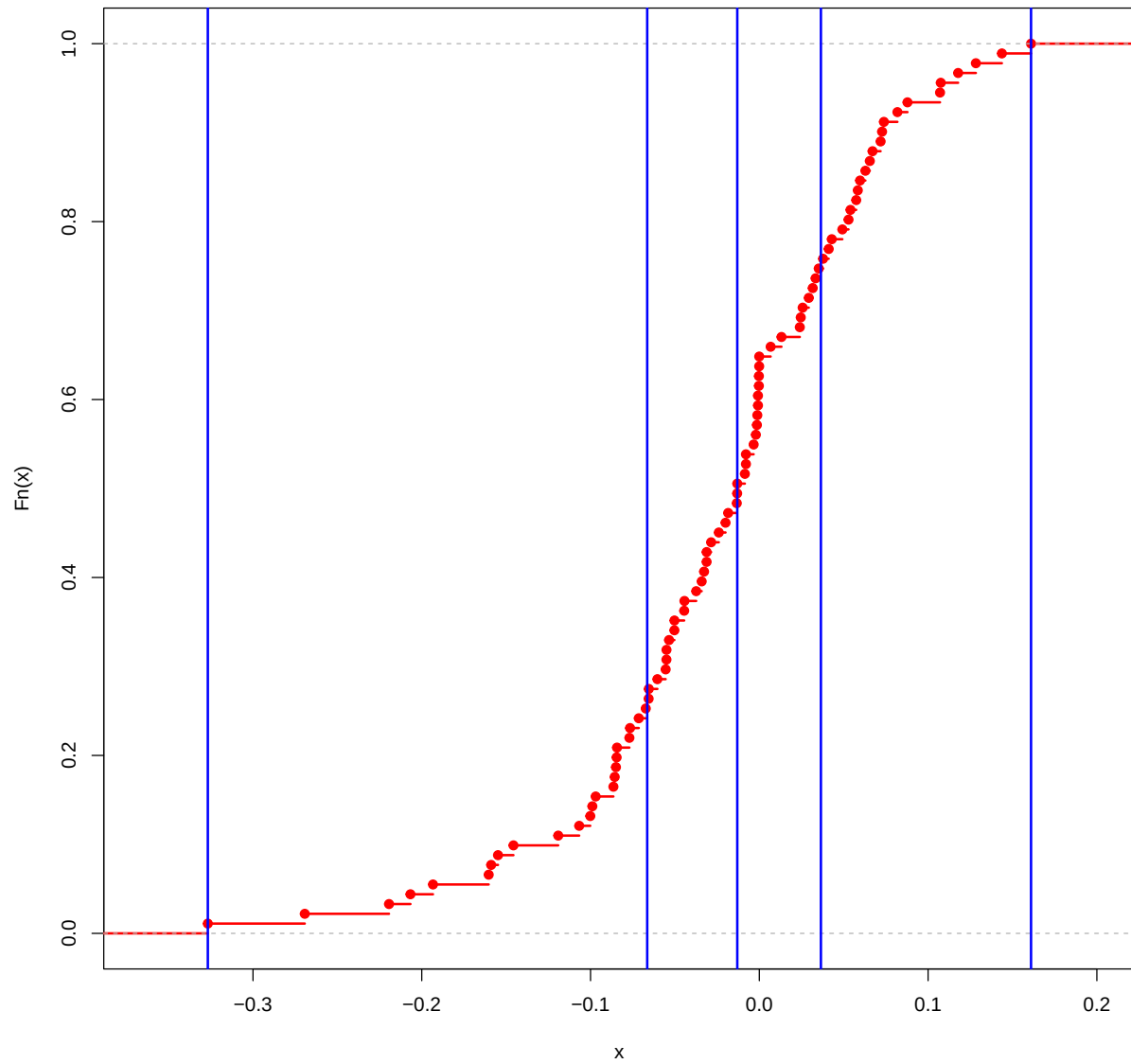


Figure 15: Residual EDF Plot, Metric, $r = 5$

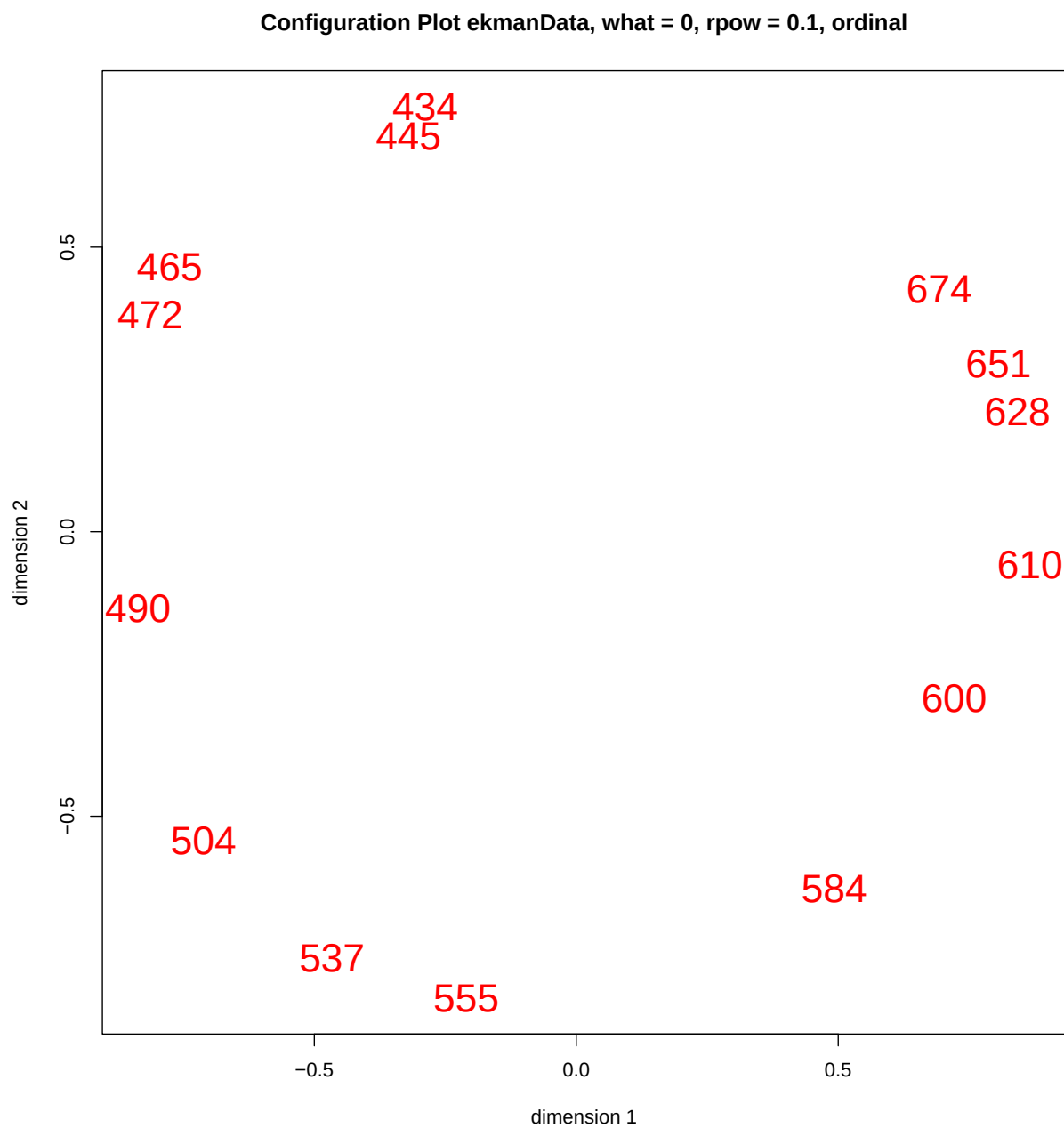


Figure 16: Configuration Plot, Non-metric, $r = .1$

ShepardPlot ekmanData, what = 0, rpow = 0.1, ordinal

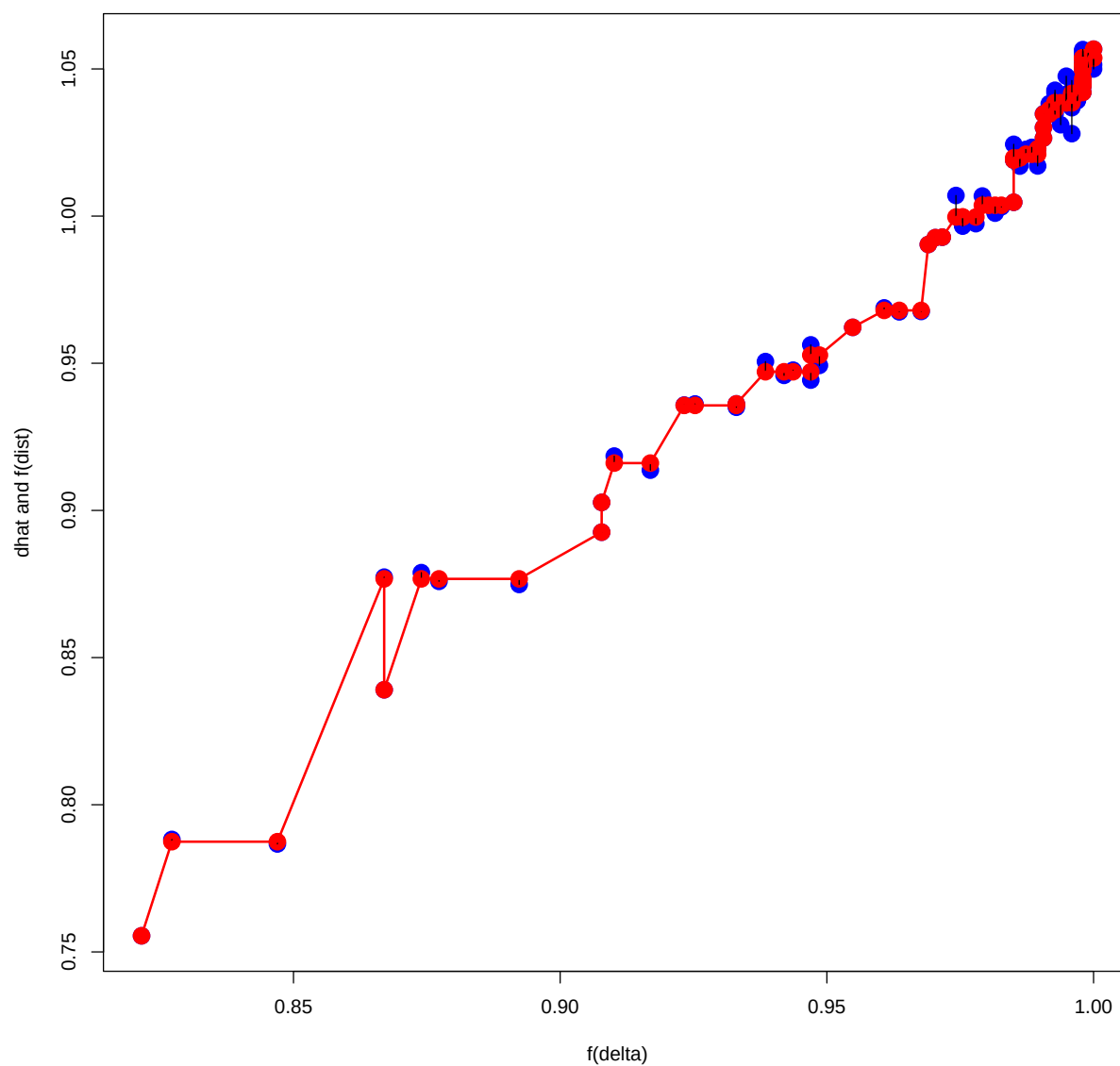


Figure 17: Shepard Plot, Non-metric, $r = .1$

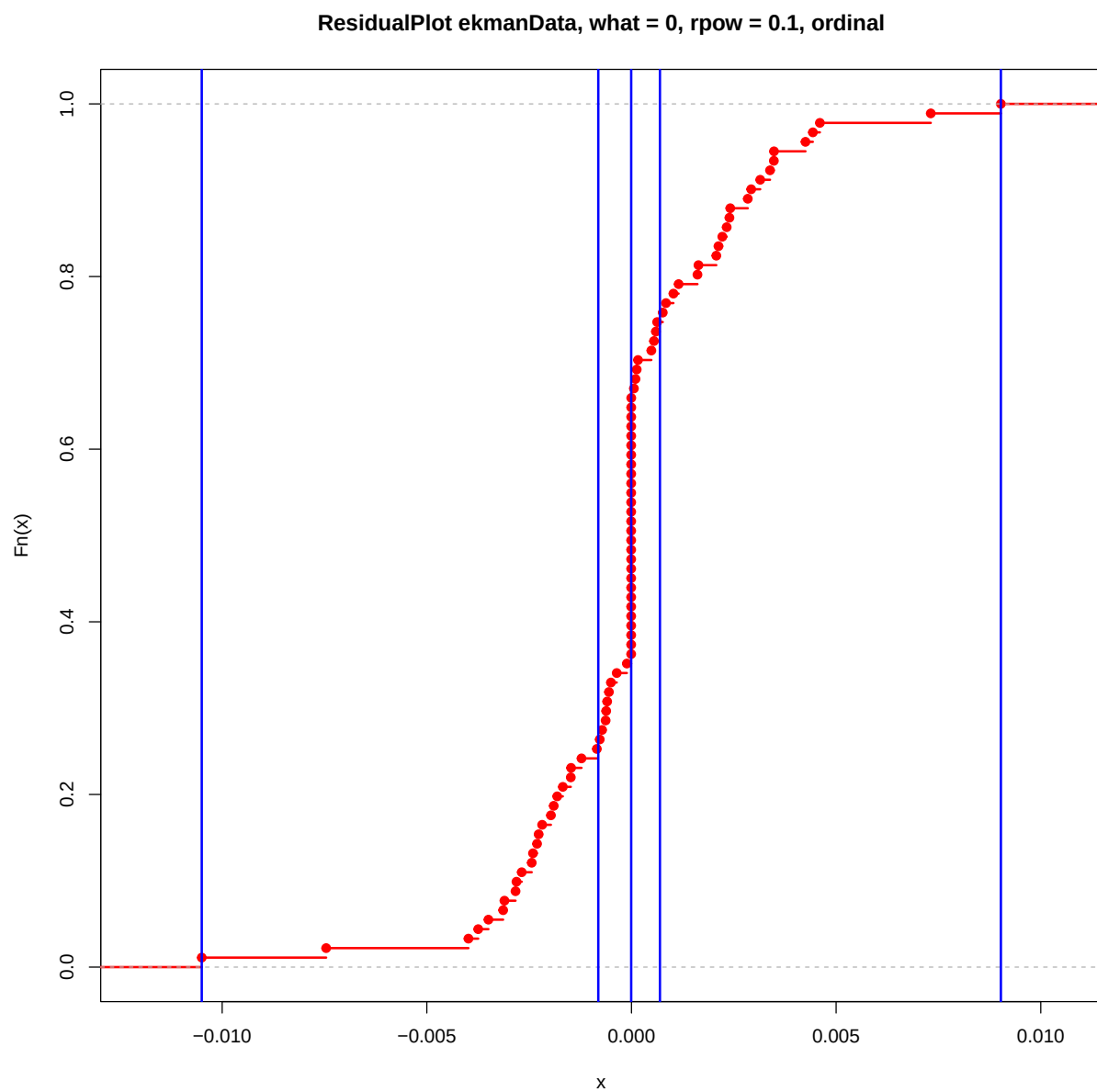


Figure 18: Residual EDF Plot, Non-metric, $r = .1$

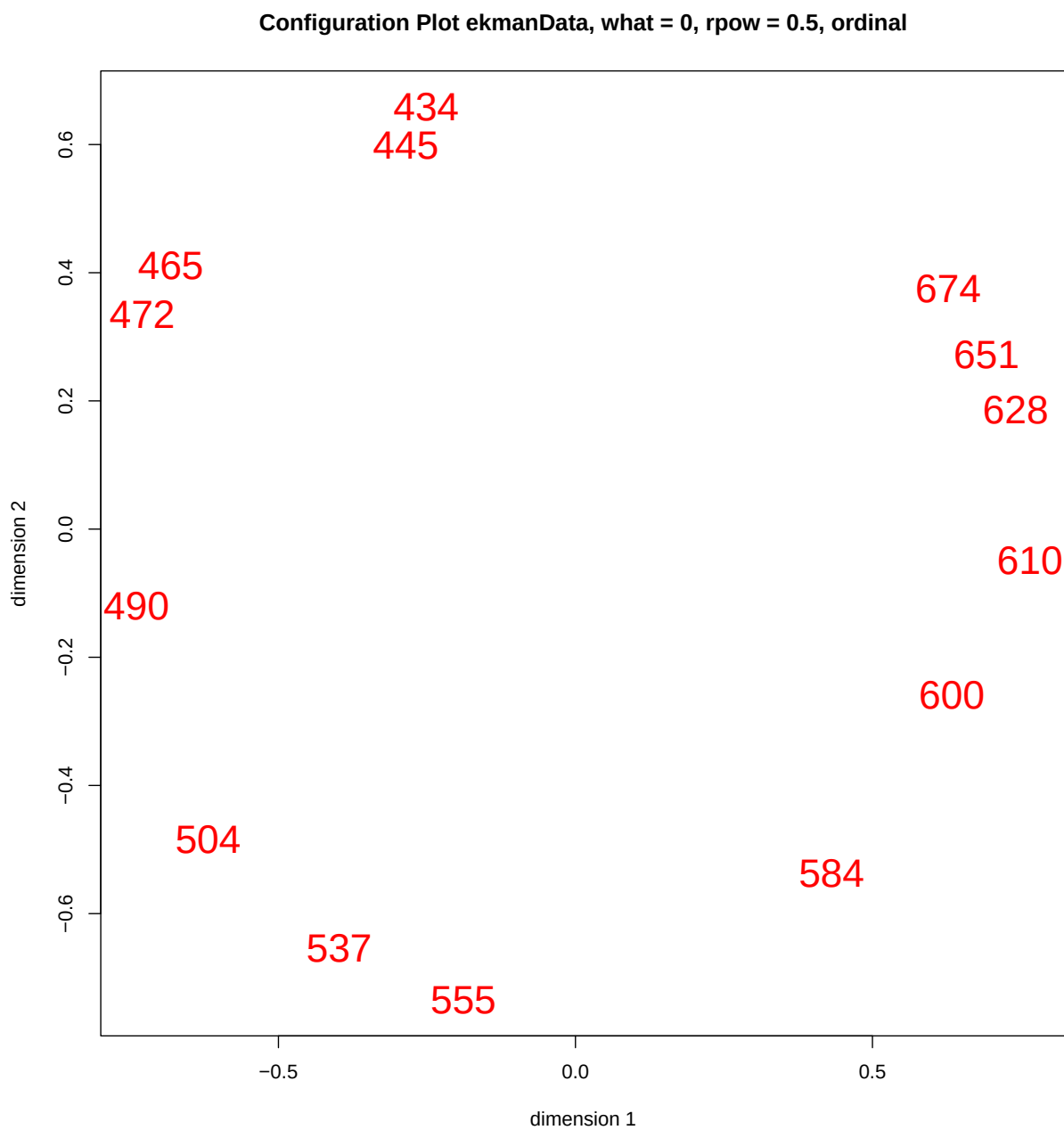


Figure 19: Configuration Plot, Non-metric, $r = .5$

ShepardPlot ekmanData, what = 0, rpow = 0.5, ordinal

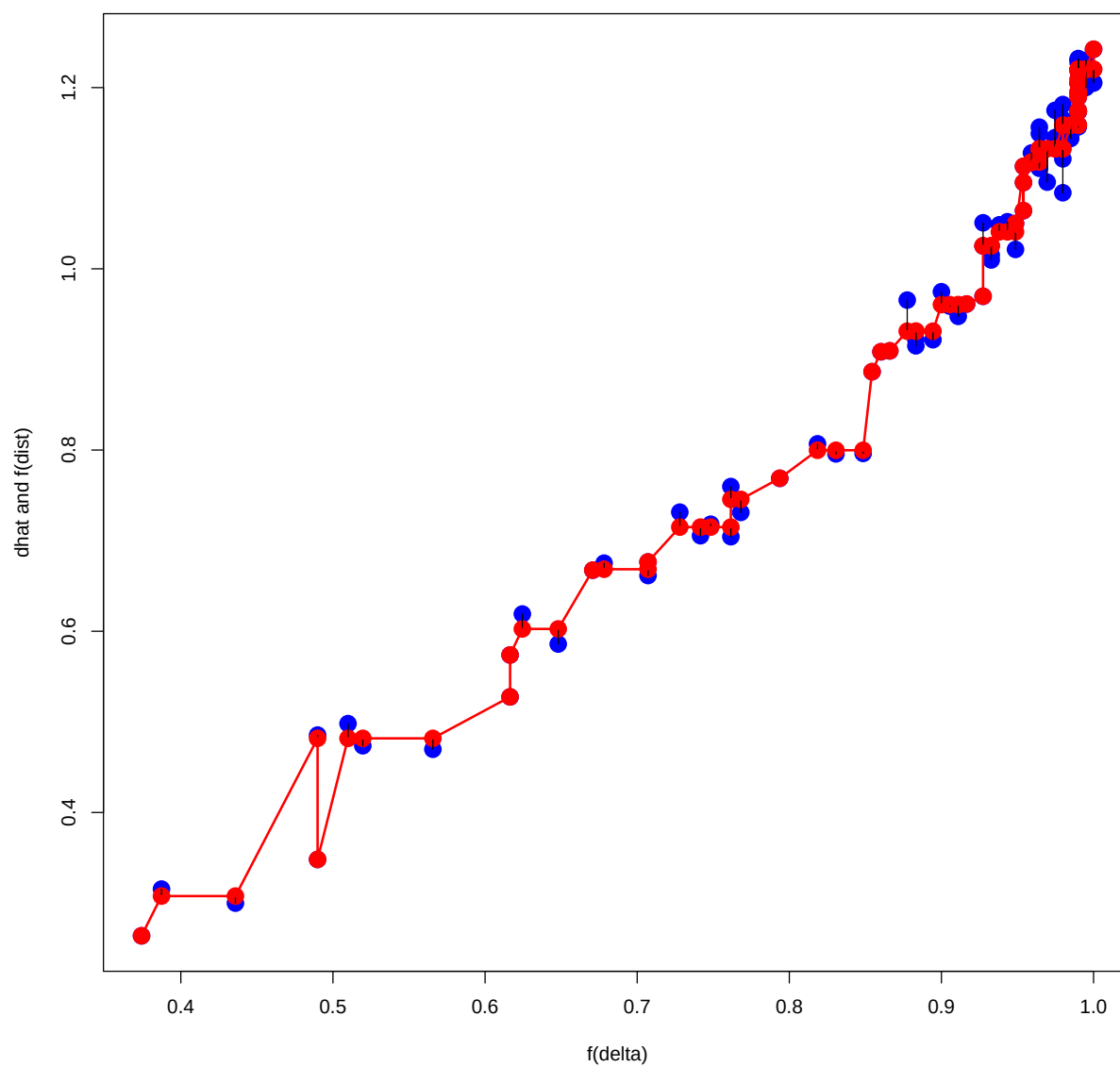


Figure 20: Shepard Plot, Non-metric, $r = .5$

ResidualPlot ekmanData, what = 0, rpow = 0.5, ordinal

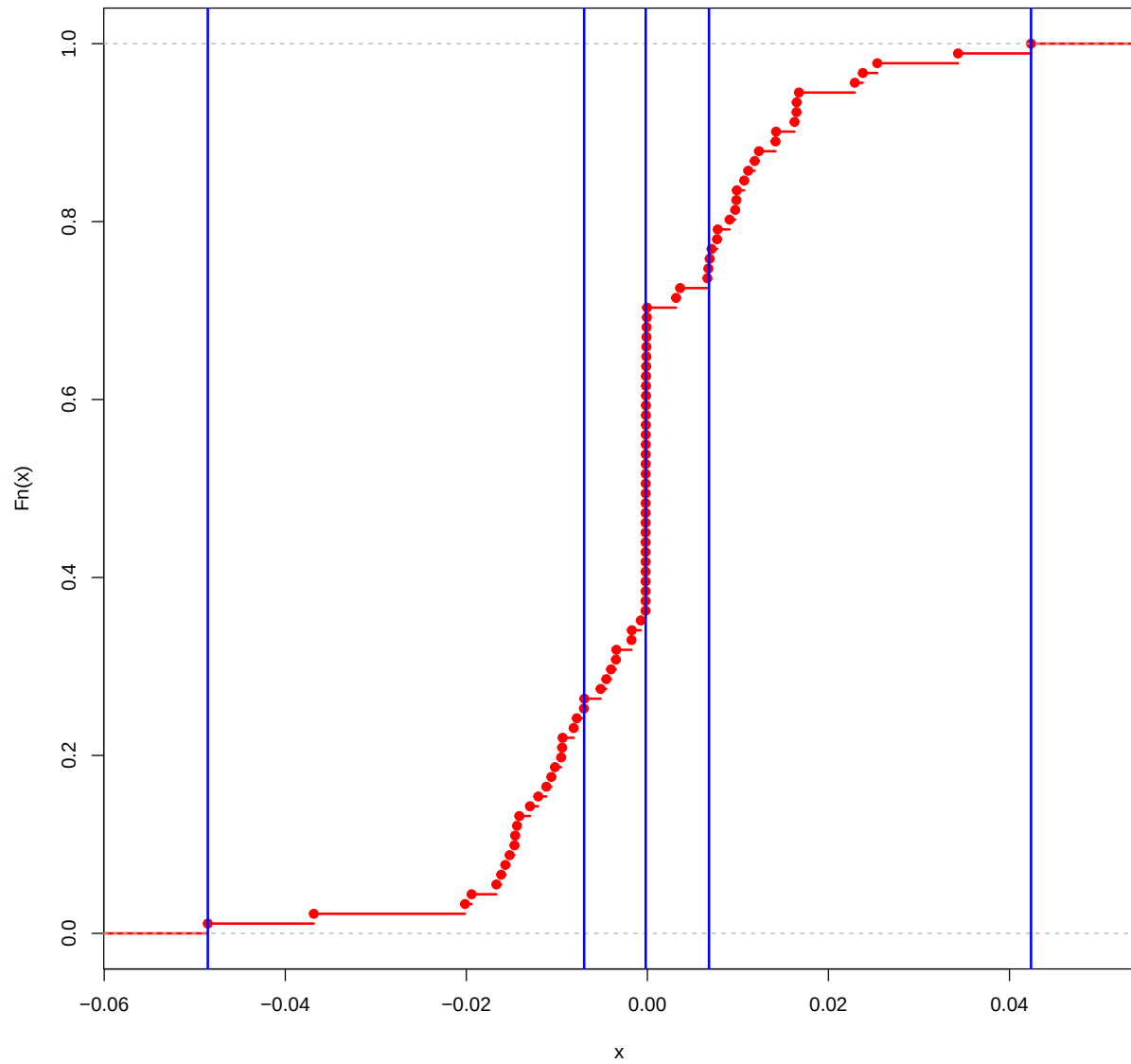


Figure 21: Residual EDF Plot, Non-metric, $r = .5$

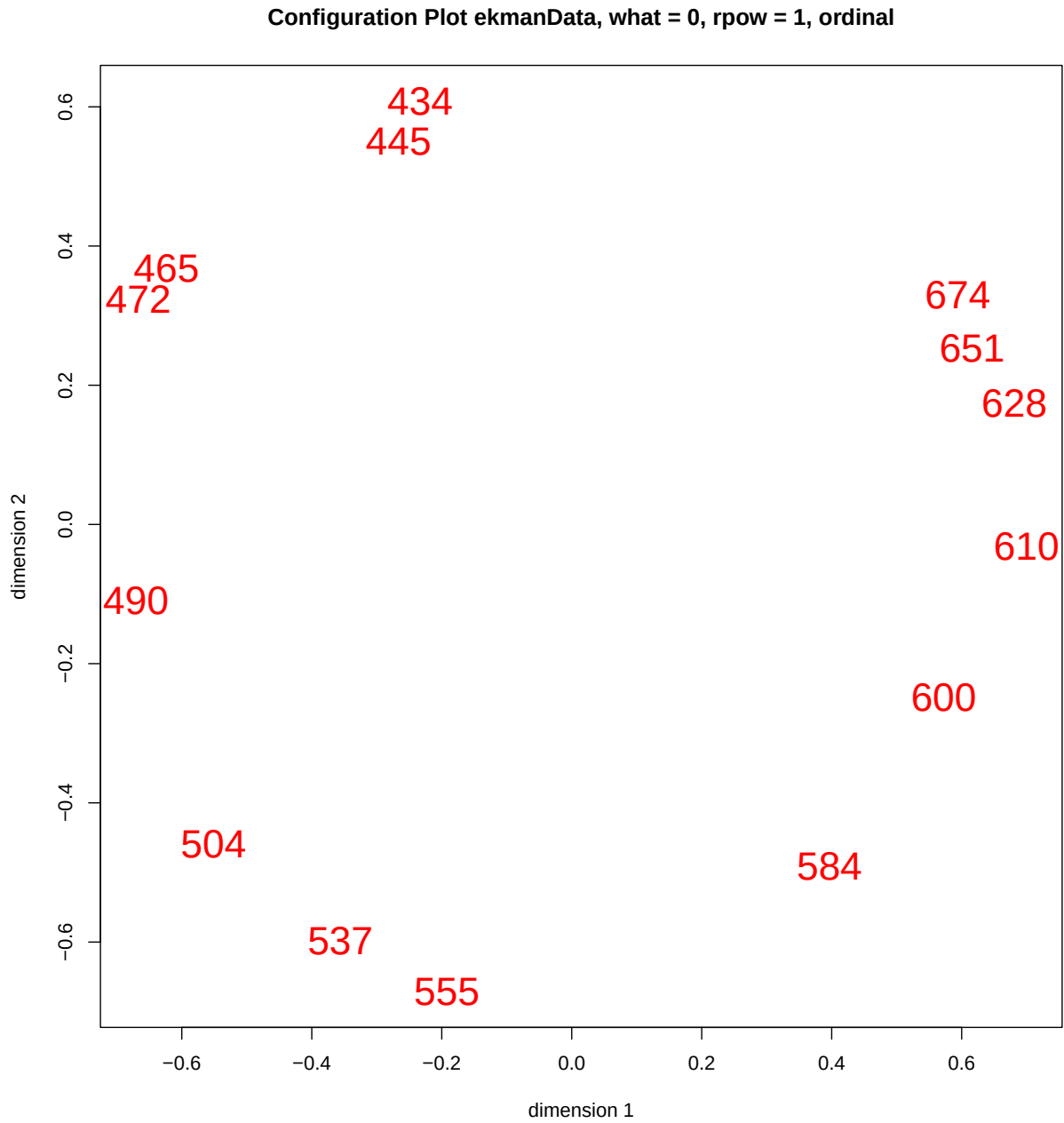


Figure 22: Configuration Plot, Non-metric, $r = 1$

ShepardPlot ekmanData, what = 0, rpow = 1, ordinal

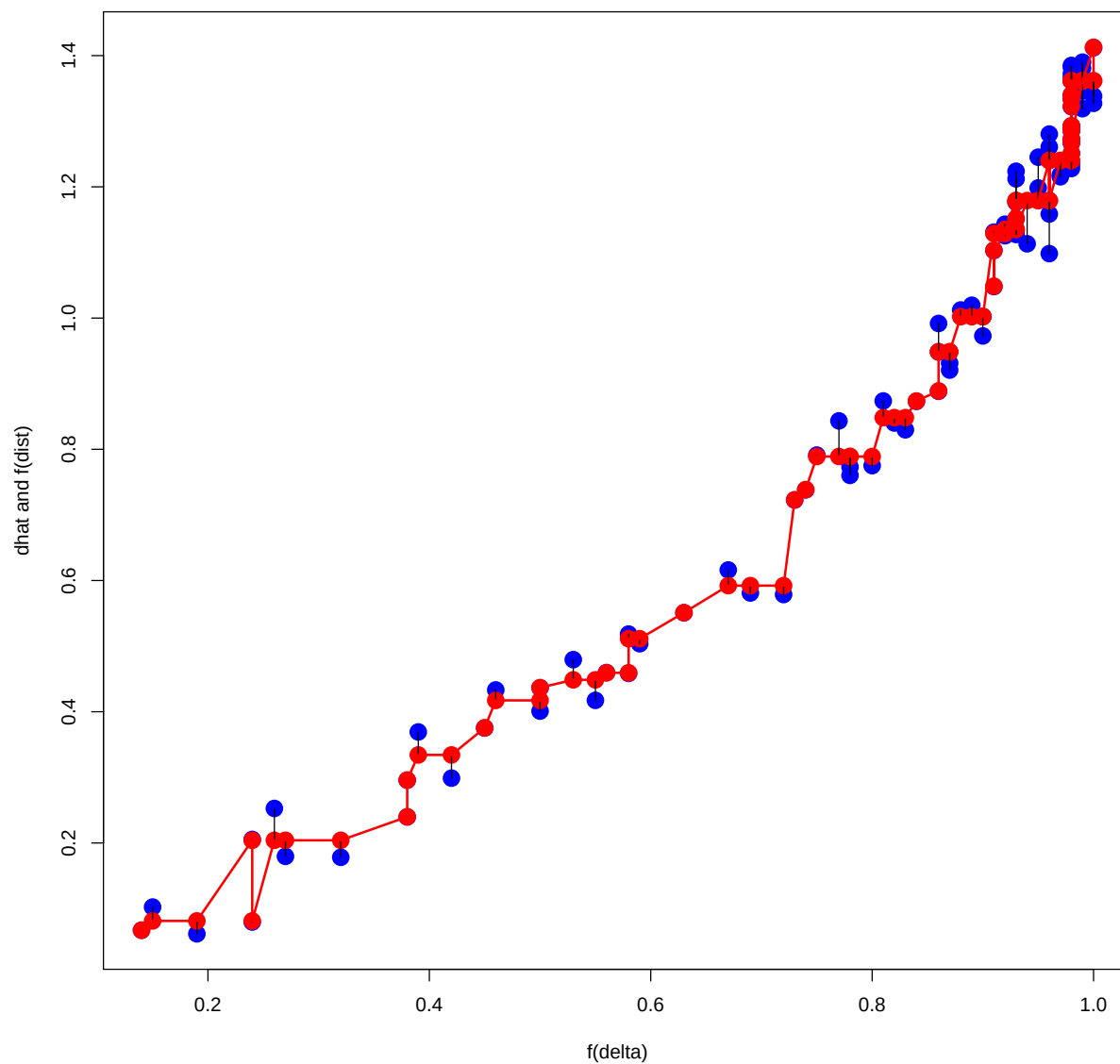


Figure 23: Shepard Plot, Non-metric, $r = 1$

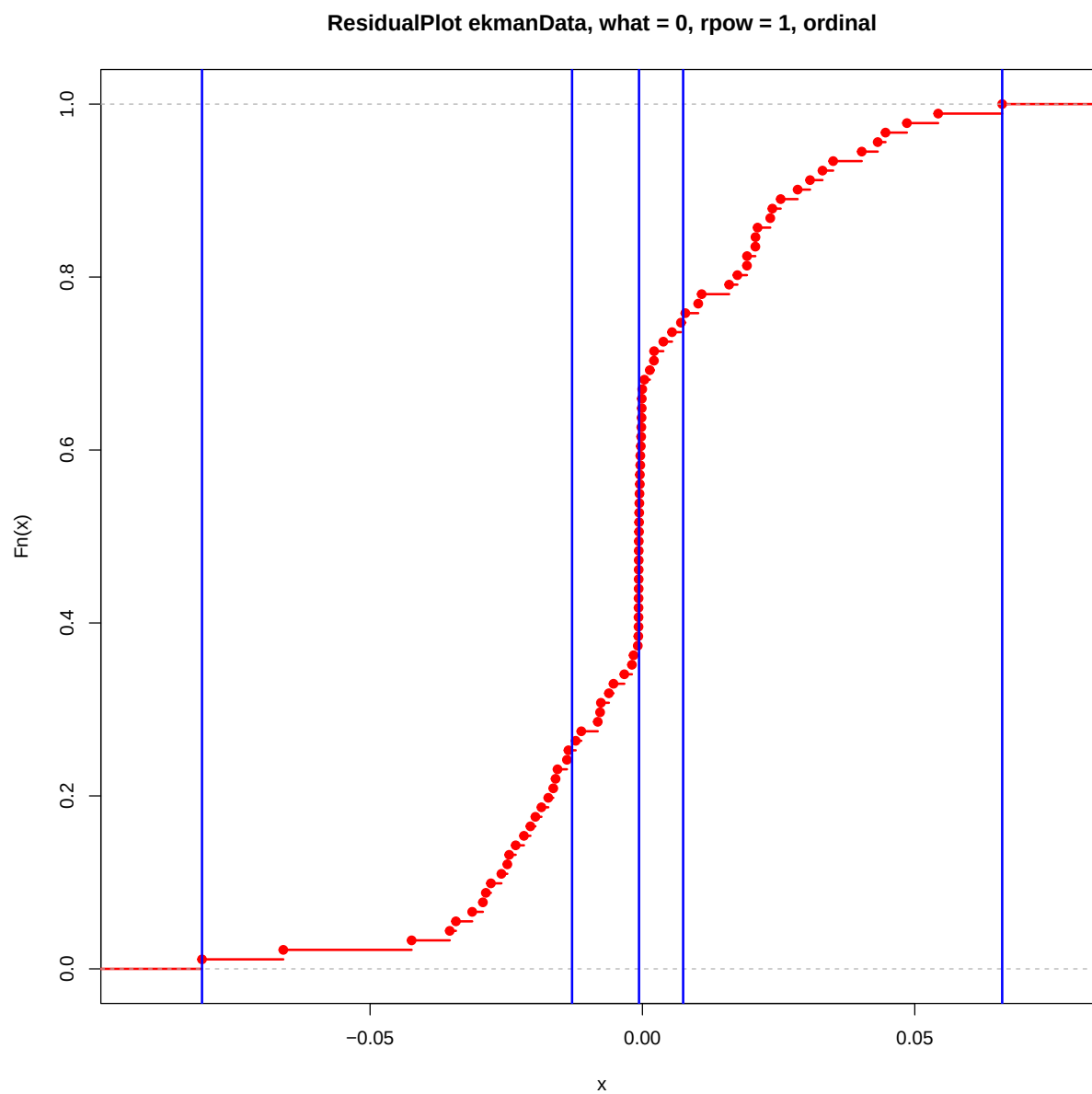


Figure 24: Residual EDF Plot, Non-metric, $r = 1$

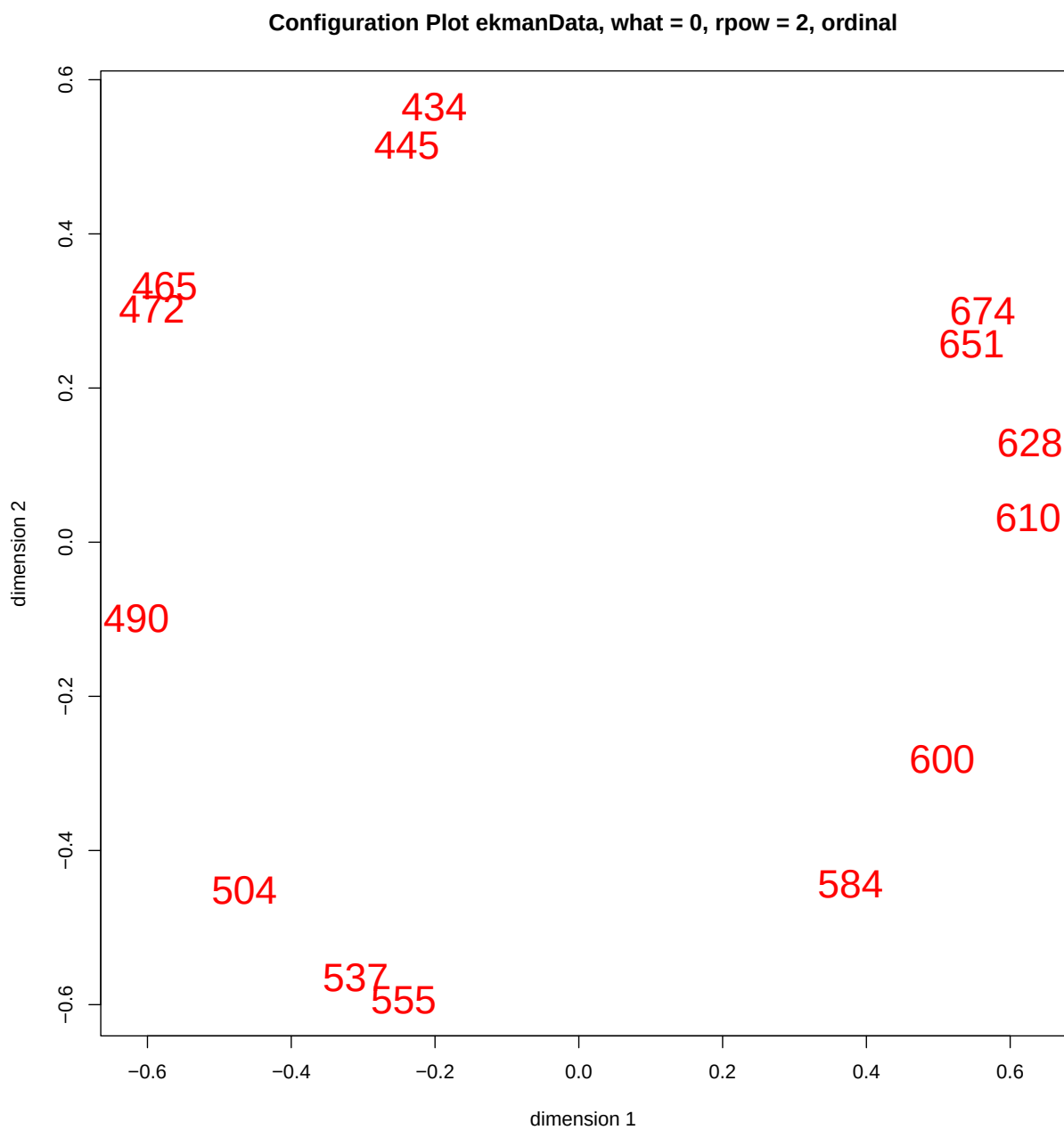


Figure 25: Configuration Plot, Non-metric, $r = 2$

ShepardPlot ekmanData, what = 0, rpow = 2, ordinal

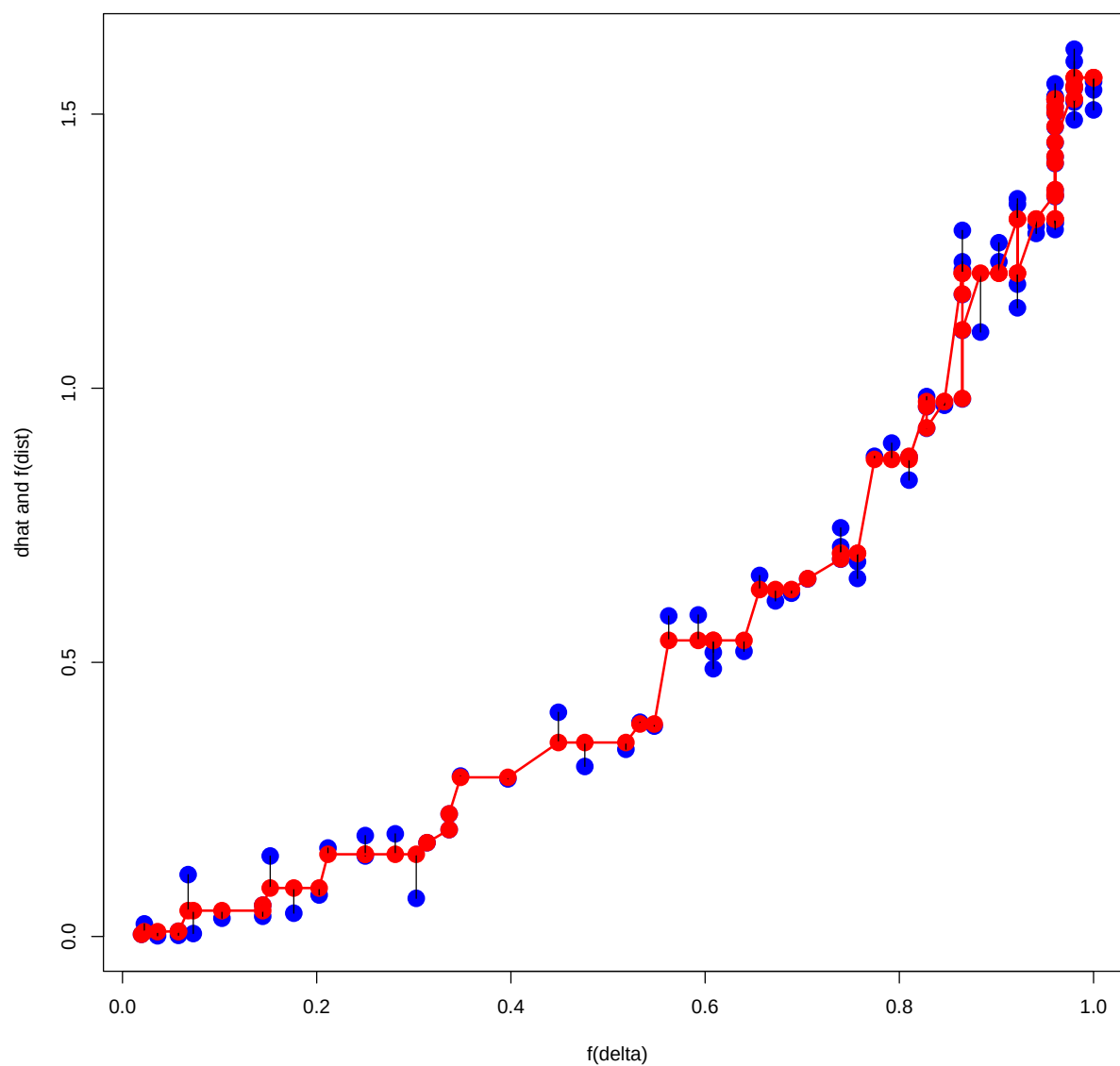


Figure 26: Shepard Plot, Non-metric, $r = 2$

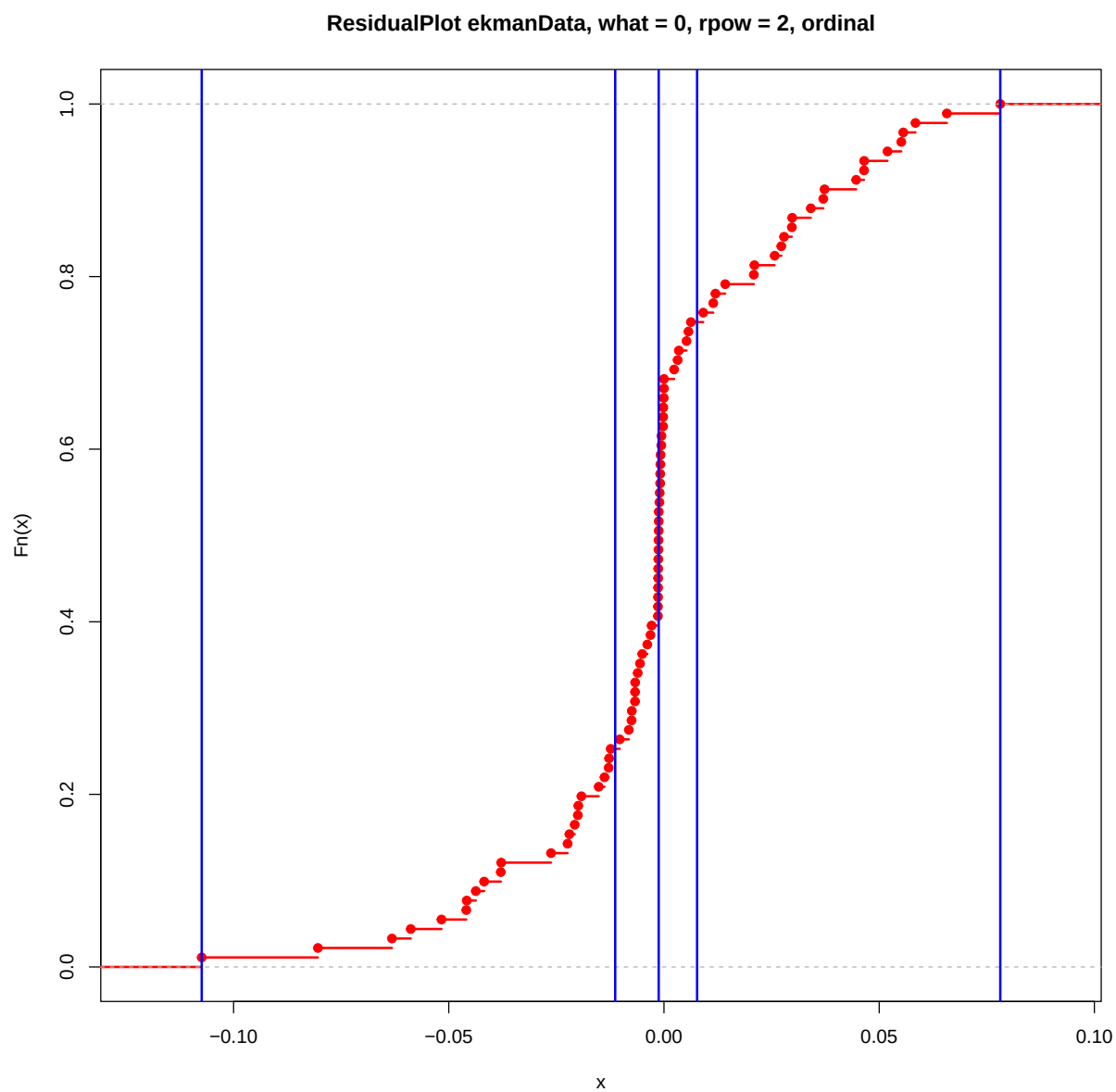


Figure 27: Residual EDF Plot, Non-metric, $r = 2$

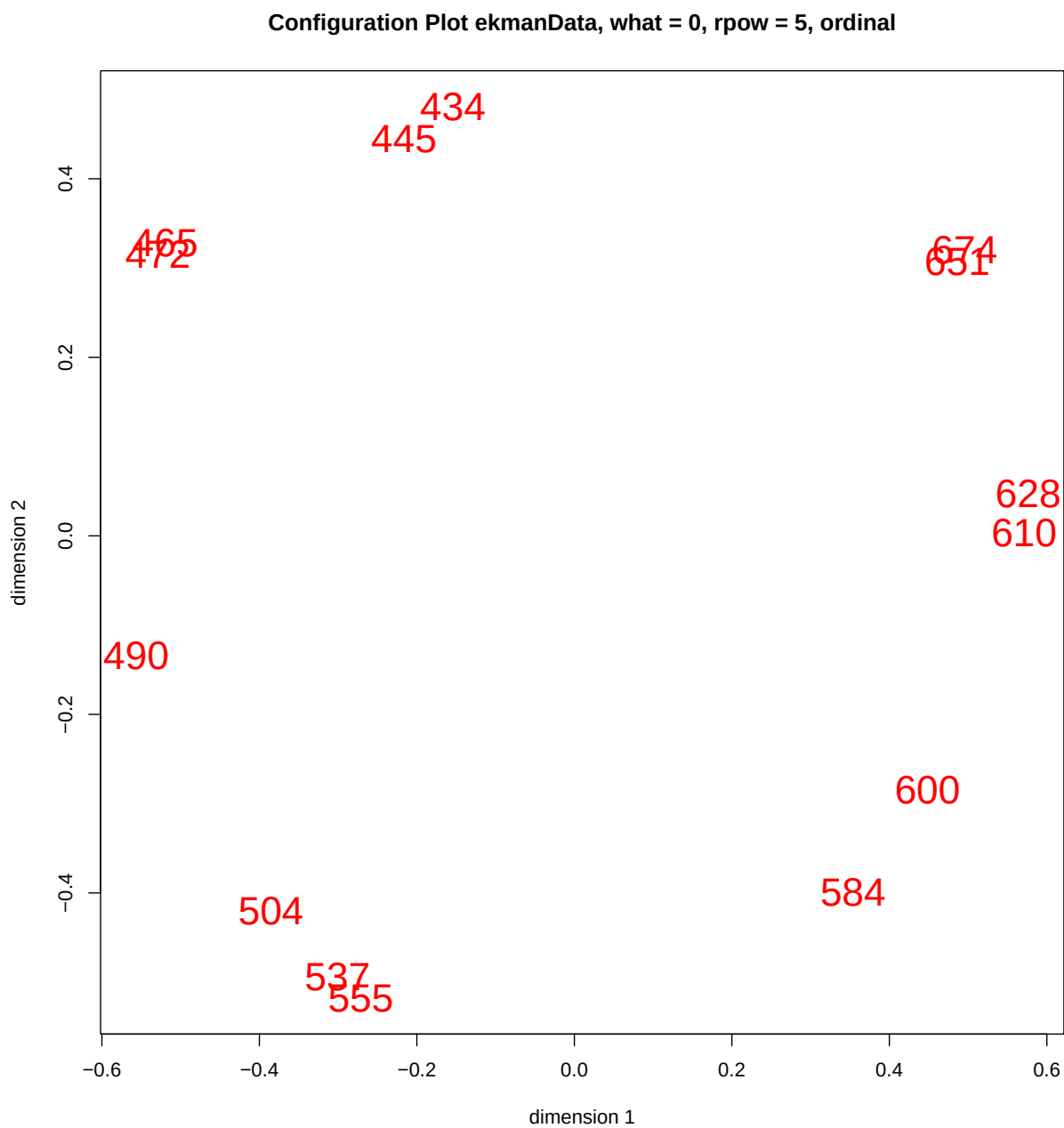


Figure 28: Configuration Plot, Non-metric, $r = 5$

ShepardPlot ekmanData, what = 0, rpow = 5, ordinal

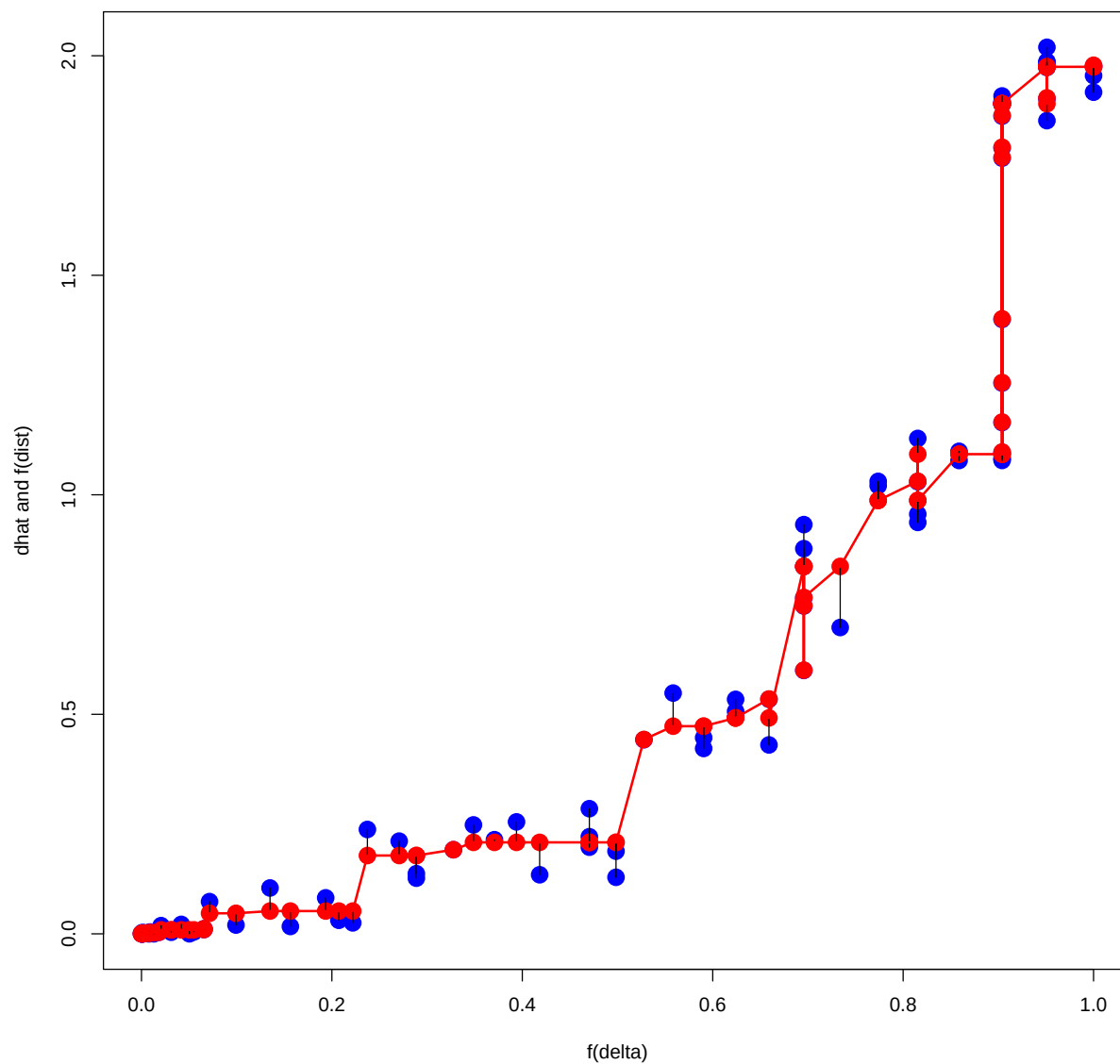


Figure 29: Shepard Plot, Non-metric, $r = 5$

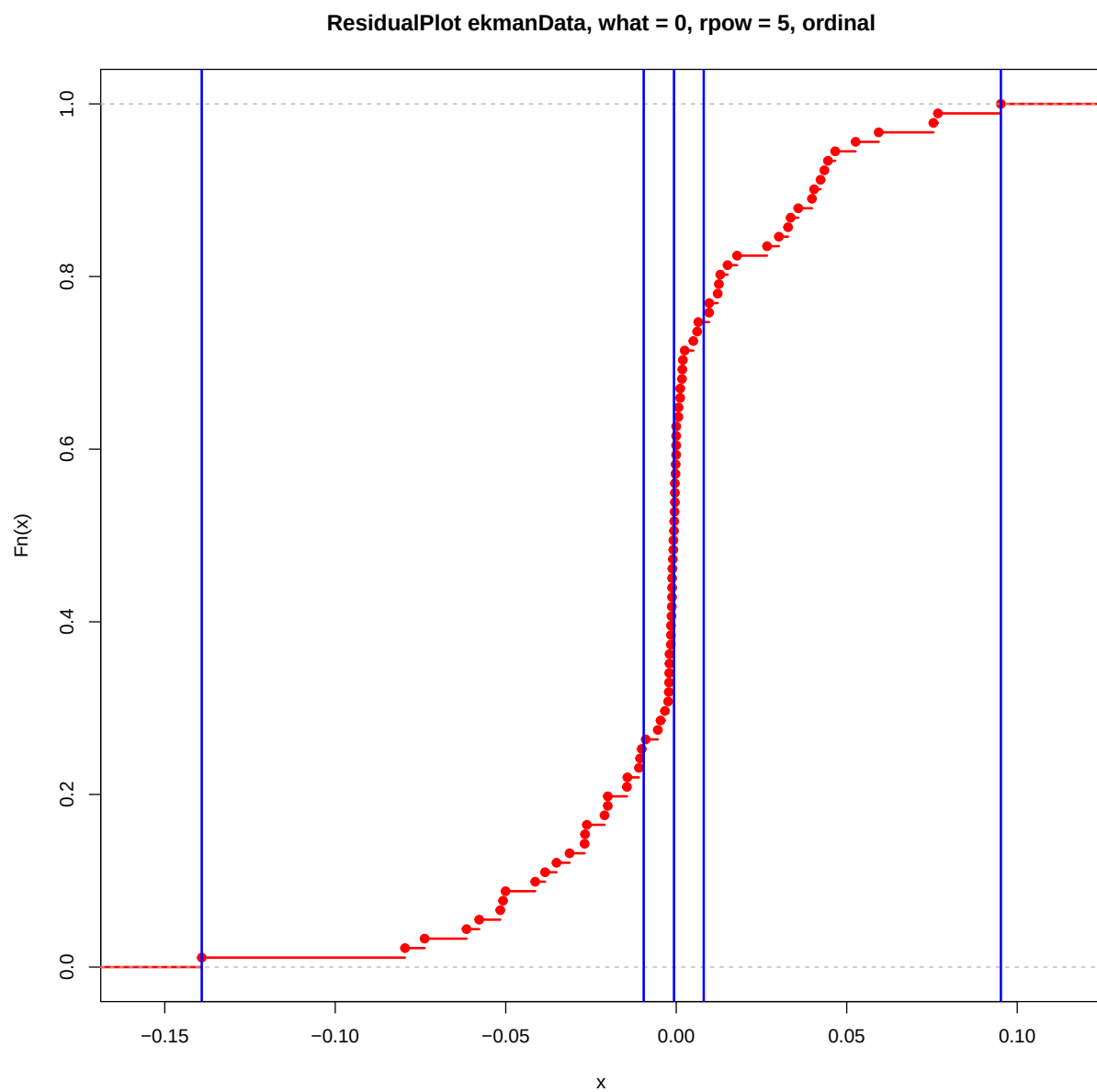


Figure 30: Residual EDF Plot, Non-metric, $r = 5$

References

- Bauschke, Heinz H., Minh N. Bui, and Xianfu Wang. 2018. “Projecting onto the Intersection of a Cone and a Sphere.” *SIAM Journal on Optimization* 28: 2158–88.
- Busing, Frank M. T. A. 2022. “Monotone Regression: A Simple and Fast $O(n)$ PAVA Implementation.” *Journal of Statistical Software* 102 (Code Snippet 1). <https://www.jstatsoft.org/index.php/jss/article/view/v102c01/4306>.
- De Leeuw, Jan. 1975. “A Normalized Cone Regression Approach to Alternating Least Squares Algorithms.” Department of Data Theory FSW/RUL. <https://jansweb.netlify.app/publication/deleeuw-u-75-a/deleeuw-u-75-a.pdf>.
- De Leeuw, Jan. 1977. “Applications of Convex Analysis to Multidimensional Scaling.” In *Recent Developments in Statistics*, edited by J. R. Barra, F. Brodeau, G. Romier, and B. Van Cutsem. North Holland Publishing Company.
- De Leeuw, Jan. 1984. “Differentiability of Kruskal’s Stress at a Local Minimum.” *Psychometrika* 49: 111–13.
- De Leeuw, Jan. 2017. “Higher Partial of fStress. Who Needs Them ?” <https://jansweb.netlify.app/publication/deleeuw-e-17-r/deleeuw-e-17-r.pdf>.
- De Leeuw, Jan. 2025. “Yet Another Smacof - Square Symmetric Case.” <https://jansweb.netlify.app/publication/deleeuw-e-25-b/>.
- De Leeuw, Jan, Patrick Groenen, and Patrick Mair. 2016. “Minimizing rStress Using Majorization.” <https://jansweb.netlify.app/publication/deleeuw-groenen-mair-e-16-a/deleeuw-groenen-mair-e-16-a.pdf>.
- De Leeuw, Jan, Kurt Hornik, and Patrick Mair. 2009. “Isotone Optimization in R: Pool-Adjacent-Violators Algorithm (PAVA) and Active Set Methods.” *Journal of Statistical Software* 32 (5): 1–24.
- De Leeuw, Jan, and Patrick Mair. 2009. “Multidimensional Scaling Using Majorization: SMACOF in R.” *Journal of Statistical Software* 31 (3): 1–30. <https://www.jstatsoft.org/article/view/v031i03>.
- De Leeuw, Jan, and Patrick Mair. 2015. “Shepard Diagram.” In *Wiley StatsRef: Statistics Reference Online*. Wiley.

- Ekman, Gosta. 1954. "Dimensions of Color Vision." *Journal of Psychology* 38: 467–74.
- Groenen, Patrick J. F., Jan De Leeuw, and Rudolf Mathar. 1995. "Least Squares Multidimensional Scaling with Transformed Distances." In *From Data to Knowledge: Theoretical and Practical Aspects of Classification, Data Analysis and Knowledge Organization*, edited by W. Gaul and D. Pfeifer. Springer Verlag. <https://jansweb.netlify.app/publication/groenen-deleeuw-mathar-c-95/groenen-deleeuw-mathar-c-95.pdf>.
- Groenen, Patrick J. F., and Michel Van de Velden. 2016. "Multidimensional Scaling by Majorization: A Review." *Journal of Statistical Software* 73 (8): 1–26. <https://www.jstatsoft.org/index.php/jss/article/view/v073i08>.
- Healy, Michael. 1968. "Algorithm AS 7: Inversion of a Positive Semi-Definite Symmetric Matrix." *Applied Statistics* 17 (2): 198–99.
- Heiser, Willem J. 1991. "A Generalized Majorization Method for Least Squares Multidimensional Scaling of Pseudodistances that May Be Negative." *Psychometrika* 56 (1): 7–27.
- Kruskal, Joseph B. 1964a. "Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis." *Psychometrika* 29: 1–27.
- Kruskal, Joseph B. 1964b. "Nonmetric Multidimensional Scaling: a Numerical Method." *Psychometrika* 29: 115–29.
- Mair, Patrick, Patrick J. F. Groenen, and Jan De Leeuw. 2022. "More on Multidimensional Scaling in R: smacof Version 2." *Journal of Statistical Software* 102 (10): 1–47. <https://www.jstatsoft.org/article/view/v102i10>.
- Malone, Samuel W., Pablo Tarazaga, and Michael W. Trosset. 2002. "Better Initial Configurations for Metric Multidimensional Scaling." *Computational Statistics and Data Analysis* 41: 143–56.
- Mersmann, Olaf. 2024. *microbenchmark: Accurate Timing Functions*. <https://CRAN.R-project.org/package=microbenchmark>.
- Qiu, Yixuan, and Jiali Mei. 2024. *RSpectra: Solvers for Large-Scale Eigenvalue and SVD Problems*. <https://CRAN.R-project.org/package=RSpectra>.
- Ramsay, James O. 1977. "Maximum Likelihood Estimation in Multidimensional Scaling." *Psychometrika* 42: 241–66.

Rusch, Thomas, Jan De Leeuw, Patrick Mair, and Kurt Hornik. In Press. “Flexible Multidimensional Scaling with the r Package Smacofx.” *Journal of Statistical Software*, In Press. <https://jansweb.netlify.app/publication/rusch-deleeuw-mair-hornik-a-25/rusch-deleeuw-mair-hornik-a-25.pdf>.

Takane, Yoshio, Forrest W. Young, and Jan De Leeuw. 1977. “Nonmetric Individual Differences in Multidimensional Scaling: An Alternating Least Squares Method with Optimal Scaling Features.” *Psychometrika* 42: 7–67.